



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Лазар Нагулов

DSL и LSP за генерисање тест података

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Лазар Нагулов	Број индекса:	SV61/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

DSL и LSP за генерисање тест података

ТЕКСТ ЗАДАТКА:

Пројектовати и имплементирати екстерни језик специфичан за домен (DSL) са декларативном синтаксом за потребе стохастичког генерисања тест података уз очување референцијалног интегритета. Реализовати наменски преводилац у програмском језику <i>Rust</i>, који обухвата фазе лексичке, синтаксне и семантичке анализе, као и трансформацију у високоинивојску међурепрезентацију (HIR). Омогућити подршку за модуларни систем са засебним превођењем и бинарном серијализацијом модула. Развити пратећи језички сервер заснован на Протоколу језичких сервера (LSP) ради интеграције са савременим развојним окружењима и пружања дијагностике у реалном времену. Спровести емпиријску евалуацију перформанси превођења, скалабилности модула, токовне генерације података и времена одзива језичког сервера.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Лазар Нагулов
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	DSL и LSP за генерисање тест података
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/62/47/6/44/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	генерисање тест података, језик специфичан за домен, протокол језичких сервера, преводац, <i>Rust</i> , статичка анализа, стохастички генератор.
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	У раду су представљени дизајн и имплементација екстерног језика специфичног за домен (DSL) за генерисање тест података. Имплементиран је преводац у програмском језику <i>Rust</i> , са лексичком, синтаксном и семантичком анализом и модуларном архитектуром заснованом на високонивојској међурепрезентацији (HIR). Генератор подржава статистичке расподеле и очување референцијалног интегритета. Реализован је и језички сервер (LSP) за статичку анализу и допуњавање кода. Мерење перформанси показује потрошњу меморије од 2 MB и време извршавања од 3,63 s при генерисању 500.000 записа. Решење постиже краће време извршавања од референтних имплементација у <i>Python</i> -у и <i>JavaScript</i> -у и испуњава критеријуме интерактивног рада за датотеке до 100.000 линија кода.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	23.09.2026
Чланови комисије, КО:	Председник: Др Никола Лубурић, ванредни професор
	Члан: Др Милан Стојков, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Lazar Nagulov
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	DSL and LSP for test data generation
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/62/47/6/44/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	test data generation, domain-specific language, Language Server Protocol, compiler, Rust, static analysis, stochastic generation.
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents the design and implementation of an external Domain-Specific Language (DSL) for test data generation. The toolchain includes a compiler implemented in Rust that performs lexical, syntactic, and semantic analysis, with a modular architecture based on a High-Level Intermediate Representation (HIR). The generator supports statistical distributions and preserves referential integrity. A Language Server Protocol (LSP) server provides static analysis and code completion. Performance evaluation shows a memory footprint of 2 MB and an execution time of 3.63 s for generating 500,000 records. The solution achieves shorter execution times than reference implementations in Python and JavaScript and meets interactive performance requirements for files of up to 100,000 lines of code.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	23.09.2026
Defended Board, DB :	President: Nikola Luburić, Phd., assoc. professor
	Member: Milan Stojkov, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	1
1.2	Предмет и циљ рада	1
1.3	Допринос рада	1
1.4	Структура рада	1
2	Теоријске основе	3
2.1	Језици специфични за домен	3
2.2	Фазе обраде програмског језика	4
2.3	Међурепрезентације	5
2.4	Систем модула	7
2.5	Генерисање података	9
2.6	Протокол језичких сервера	11
2.7	Архитектура командних интерфејса и рутирање захтева	12
3	Анализа постојећих решења	15
3.1	Језици специфични за домен	15
3.2	Системи за дефинисање шема	16
3.3	Системи за генерисање тест података	17
3.4	Закључна разматрања и упоредни приказ	18
4	Дизајн решења	19
4.1	Архитектура система	19
4.2	Дизајн језика специфичног за домен	20
4.3	Дизајн семантичке анализе	22
4.4	Дизајн високонивојске међурепрезентације	22
4.5	Дизајн модула	23
4.6	Дизајн генератора података	24
4.7	Дизајн језичког сервера	25
4.8	Дизајн командног интерфејса	26
4.9	Изазови и проблеми	26
5	Имплементација	29
5.1	Лексичка анализа	29
5.2	Синтаксна анализа	29
5.3	Апстрактно синтаксно стабло	31
5.4	Семантичка анализа	32
5.5	Високонивојска међурепрезентација	34
5.6	Систем модула	35
5.7	Генерисање података	36
5.8	Језички сервер	36
5.9	Командни интерфејс	37
6	Евалуација и дискусија	39
6.1	Експериментално окружење	39
6.2	Евалуација процеса превођења	39
6.3	Евалуација система модула	40
6.4	Евалуација генерисања података	41
6.5	Евалуација језичког сервера	43

6.6 Дискусија резултата	44
7 Закључак	45
8 Будући рад	47
8.1 Унапређење система превођења	47
8.2 Средњенивојска међурепрезентација	47
Списак слика	49
Списак листинга	51
Списак табела	53
Списак коришћених скраћеница	55
Списак коришћених појмова	57
Биографија	59
Литература	61

Осигурање квалитета у развоју софтверских система ослања се на континуирано тестирање. Аутоматизовано тестирање захтева скупове података који поштују структурна и релациона ограничења. Приступи генерисању података за тестирање ослањају се на писање императивног програмског кода или коришћење графичких корисничких интерфејса (енг. *Graphical User Interface*, GUI). Језици специфични за домен (енг. *Domain-Specific Language*, DSL) нуде алтернативу кроз висок ниво апстракције, омогућавајући инжењерима фокусирање на структуру података уместо на алгоритам њихове синтезе [1]. Успешна примена DSL-а зависи од квалитета пратећих развојних алата. Протокол језичких сервера (енг. *Language Server Protocol*, LSP) представља стандард који омогућава централизовани развој функција за анализу и обраду кода. LSP премошћује јаз између DSL-а и развојних окружења [2].

1.1 Мотивација

Постојећи алати за генерацију података захтевају компромис између експресивности и једноставности употребе. Писање наменских скрипти одузима време и отежава одржавање, док визуелни алати ограничавају могућност верзионисања и аутоматизације. Увођење новог DSL-а наилази на отпор програмера уколико језик не пружа помоћ при писању кода [3]. Креирање наменских додатака за свако појединачно развојно окружење захтева додатне ресурсе [4]. Стандардизација комуникације путем LSP-а омогућава развој интегрисаних система где једна имплементација језичког сервера пружа конзистентно развојно искуство, дијагностику и аутоматско допуњавање кода [5].

1.2 Предмет и циљ рада

Предмет рада чине дизајн и имплементација DSL-а намењеног генерисању тест података. Рад обухвата развој прилагођеног преводиоца (енг. *compiler*) и пратећег језичког сервера који анализира изворни код. Циљ рада је унапређење ефикасности инжењера током процеса развоја софтвера кроз пружање специјализованог алата. Развијени систем треба да омогући дефинисање модела података уз детекцију семантичких грешака у реалном времену, навигацију кроз дефиниције симбола и интелигентно допуњавање кода.

1.3 Допринос рада

Рад доноси побољшања у контексту алата за синтетизацију података кроз:

- пројектовање новог доменског језика са декларативном синтаксом који омогућава описивање модела података, њихових типова и релационих зависности,
- имплементацију модуларне архитектуре преводиоца која подржава увоз спољних датотека, чиме се омогућава степен поновне употребљивости кода и убрзава процес превођења пројеката и
- развој LSP сервера који обједињује брзину локалног генерисања података са могућностима статичке анализе кода унутар развојних окружења.

1.4 Структура рада

Рад је организован у осам поглавља. Након уводног дела, друго поглавље описује теоријске концепте преводилаца, генерисања података и архитектуре језичких сервера. Треће поглавље анализира предности и мане постојећих софтверских решења у предметном домену. Четврто поглавље приказује дизајн и архитектуру предложеног система, док пето поглавље детаљно образлаже техничку имплементацију преводилаца и пратећих сервиса. Шесто поглавље представља резултате евалуације система и анализу перформанси. Седмо поглавље доноси закључак, а осмо разматра потенцијалне правце за будућа унапређења решења.

Ово поглавље приказује теоријске концепте неопходне за разумевање архитектуре и имплементације система. Први део дефинише DSL и њихову класификацију (поглавље 2.1). Други део анализира фазе лексичке, синтаксне и семантичке анализе текста (поглавље 2.2). Трећи део дефинише концепте међуре-презентација (енг. *Intermediate Representation*, IR) у преводиоцима (поглавље 2.3). Четврти део описује теоријске основе модуларних система (поглавље 2.4), док је пети део посвећен концептима лење евалуације и алгоритмима за генерисање података (поглавље 2.5). Напоследку, шести део описује архитектуру LSP-а и пратећих стандарда (поглавље 2.6).

2.1 Језици специфични за домен

DSL је програмски језик ограничених могућности намењен решавању проблема у конкретној области [1]. Синтакса и семантика DSL-а прилагођене су конкретном домену [1]. Дефиниција обухвата четири елемента:

- природа рачунарског програмског језика за давање инструкција рачунару [1]. Структура језика омогућава људима разумевање изворног кода уз задржавање извршности на рачунару [1];
- флуидност изражавања кроз међусобно комбиновање појединачних језичких конструкција [1];
- ограничена експресивност у односу на језике опште намене (енг. *General Purpose Language*, GPL) [1]. Ограничена експресивност подразумева подршку само за минималан скуп функционалности неопходних за домен [1]. Језик онемогућава развој целокупног софтверског система већ решава само један сегмент [1] и
- јасан фокус на уску област примене [1]. Фокус на домен произилази директно из ограничене могућности изражавања језика [1].

Инжењерство програмских језика прави разлику између DSL и GPL [3]. GPL пружају широк спектар могућности за решавање произвољних софтверских проблема [3]. DSL међају општу употребљивост за висок ниво апстракције и већу продуктивност унутар изабране области [3].

Разликују се интерни (енг. *internal*) и екстерни (енг. *external*) доменски језици [1], [3]. Интерни доменски језици реализују се унутар постојећег језика домаћина [1]. Синтакса интерних језика дефинисана је и ограничена синтаксним правилима изабраног језика домаћина [3]. Екстерни доменски језици поседују независну синтаксу у односу на друге програмске језике [1]. Развој екстерног језика захтева имплементацију засебног лексичког и синтаксног анализатора за анализу изворног текста [3]. Контрола над синтаксом омогућава креирање оптималних текстуалних конструкција за крајњег корисника [3].

2.1.1 Предности и изазови примене

Примена доменских језика унапређује процес развоја софтвера кроз прилагођавање синтаксе специфичном домену [3]. Висок ниво апстракције омогућава доменским експертима разумевање и валидацију логике кода [1]. Употреба специјализоване терминологије смањује учесталост семантичких грешака приликом дефинисања захтева [3]. Усклађеност концепата језика са проблемима струке скраћује време потребно за писање системских спецификација [3].

Развој доменских језика истовремено намеће специфичне инжењерске захтеве [3]. Креирање екстерног језика повећава комплексност почетне архитектонске структуре система [1]. Имплементација захтева изградњу наменских алата попут преводиоца, језичких сервера и развојних окружења [3]. Дугорочно одржавање специјализованог екосистема захтева континуирано улагање развојних ресурса [3].

2.2 Фазе обраде програмског језика

Обрада доменског језика обухвата низ сукцесивних фаза које превode изворни код у IR спремну за извршавање [6]. Процес се дели на предњи део (енг. *frontend*) и задњи део (енг. *backend*) преводиоца. Предњи део анализира изворни текст и утврђује исправност структуралних и семантичких правила [7]. Задњи део користи припремљене структуре података за покретање логике извршавања или генерисање циљног кода [6]. Предњи део преводиоца обезбеђује заједничку основу коју могу користити клијентски алати, попут механизма за извршавање или језичких сервера.

2.2.1 Лексичка анализа

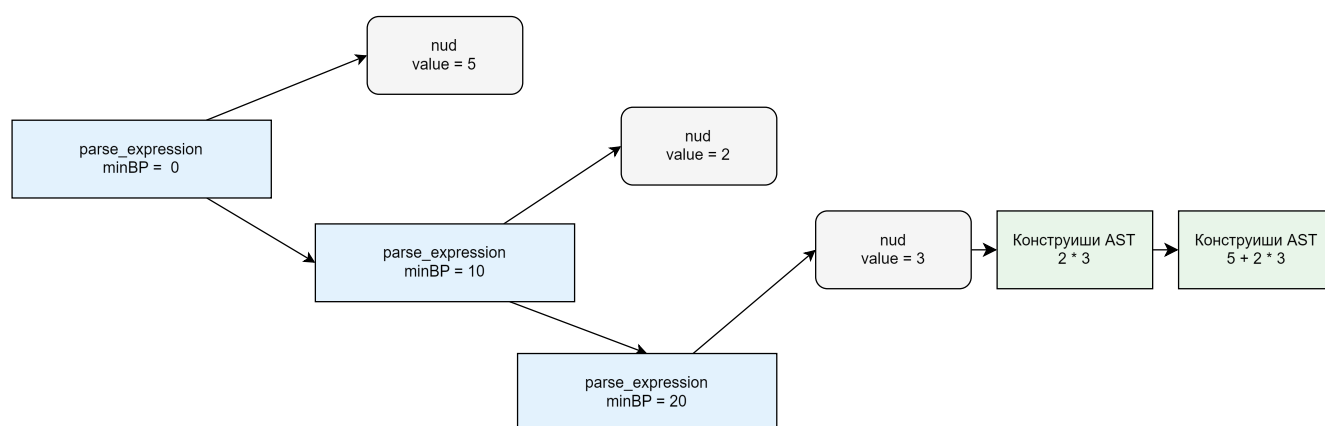
Лексичка анализа представља почетну фазу у процесу обраде изворног кода [6]. Лексички анализатор (енг. *lexer*) прихвата низ карактера из текстуалне датотеке и групише карактере у логичке целине [8]. Логичке целине називају се лексеме [6]. Свака лексема се мапира у одговарајући токен који садржи тип и вредност препознатих карактера [8]. Токени представљају кључне речи, идентификаторе, литерале и операторе језика [7]. Лексички анализатор истовремено филтрира сувишне елементе изворног кода попут размака и коментара [8]. Резултат фазе је низ токена спреман за даљу синтаксну проверу.

2.2.2 Синтаксна анализа

Синтаксна анализа утврђује структуралну исправност низа токена на основу дефинисане граматике језика [6]. Синтаксни анализатор (енг. *parser*) организује примљене токене у хијерархијску структуру података [7]. Хијерархијска структура назива се апстрактно синтаксно стабло (енг. *Abstract Syntax Tree, AST*) [8]. AST приказује операторе и операнде унутар сложених језичких конструкција [7]. Грешке откривене у процесу анализе се прослеђују кориснику или развојном окружењу путем језичког сервера.

2.2.2.1 Пратов синтаксни анализатор

Пратов синтаксни анализатор (енг. *Pratt parser*) је алгоритам синтаксне анализе заснован на првенству оператора одозго на доле [8]. Рад алгоритма почива на нумеричкој вредности која се назива снага везивања (енг. *binding power, BP*). BP експлицитно одређује ниво приоритета сваког оператора у језику [8]. Визуелни приказ трансформације линеарног низа токена у хијерархијску структуру на основу BP-а приказан је на слици 1.



Слика 1: Трансформација израза $5 + 2 * 3$ у AST на основу BP-а оператора

Токенски елементи се мапирају у две функције за обраду изворног текста:

- префиксну функцију (енг. *null denotation* или *nud*), која обрађује токене на самом почетку израза и
- инфиксну функцију (енг. *left denotation* или *led*), која обрађује токене који се појављују са десне стране већ делимично изграђеног стабла [8].

Главна петља алгоритма пореди BP тренутног оператора са BP-ом следећег токена у низу [8]. Синтаксни анализатор наставља са груписањем аргумената у подстабло докле год је BP следећег токена већа од

тренутне вредности [8]. Предност приступа је једноставност одржавања програмског кода за анализу математичких операција [7].

2.2.3 Семантичка анализа

Семантичка анализа проверава да ли изворни код поштује правила значења и контекста програмског језика [6]. Фаза користи AST за доношење одлука о исправности програма [7]. Основне активности обухватају проверу компатибилности типова података и дефинисаност коришћених идентификатора [6]. Систем мапира откривене променљиве у структуру података које се називају табеле симбола (енг. *symbol table*) [7]. Табела симбола чува информације о опсегу видљивости, типовима и особинама сваког симбола [6], [7]. Успешна семантичка анализа потврђује исправност улазног кода пре покретања наредних фаза превођења или извршавања [3].

2.3 Међурепрезентације

IR представља интерну структуру података коју преводилац користи за моделовање изворног програма између почетних и завршних фаза обраде [6]. Језички системи ретко врше директно превођење изворног кода у машински или извршни облик. Уместо тога, користи се више сукцесивних нивоа апстракције како би се логика синтаксне анализе раздвојила од семантичке провере, независне оптимизације и коначне генерације кода [9]. Зависно од блискости изворном коду или циљној архитектури, IR се категоризује у високонивојски (енг. *High-level Intermediate Representation*, HIR), средњениивојски (енг. *Medium-level Intermediate Representation*, MIR) и нисконивојски (енг. *Low-level Intermediate Representation*) [9]. Сваки слој је прилагођен специфичним захтевима фазе извршавања.

2.3.1 Апстрактно синтаксно стабло

AST чини примарни слој који осликава граматичку структуру изворног текста [7]. Резултат је фазе синтаксне анализе. За разлику од конкретног синтаксног стабла, AST издваја декоративне елементе језика као што су заграде, тачке и белине [8]. Чворови унутар структуре представљају операторе, семантичке конструкте и језичке директиве, док гране воде ка припадајућим операндима. Особине AST-а пружају основу за почетне фазе синтаксне провере и мапирање локација грешака. Директна повезаност са граматиком чини AST неефикасним за претраге глобалних веза [9], [10].

2.3.2 Интернирање стрингова

Током синтаксне анализе изворног кода, преводилац се сусреће са појавом истих текстуалних идентификатора. Чување и сукцесивно поређење стрингова у сваком чвору IR-а уводи меморијски и рачунарски трошак. Превазилажење трошка постиже се техником интернирања стрингова (енг. *string interning*) [10].

Интернирање стрингова оптимизује меморију кроз имплементацију централизованог, непроменљивог базена стрингова (енг. *string pool*). Сваки стринг се уписује у базен само једном, при чему му се додељује нумерички идентификатор (енг. *symbol* или *ID*). Фазе унутар архитектуре преводиоца уместо комплетних текстуалних података користе додељене нумеричке кључеве. Замена операција поређења текстуалних низова поређењем целих бројева убрзава семантичку анализу и изградњу форми IR [10].

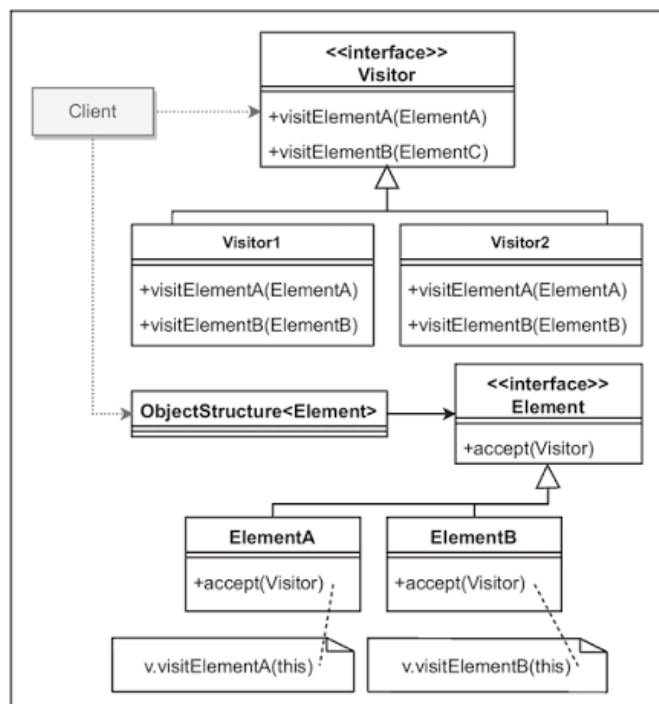
2.3.3 Дизајнерски образац посетилац

Дизајнерски обрасци су виšekратно употребљива решења за учестале проблеме у пројектовању софтвера [11]. У архитектури преводиоца, изазов представља проширење функционалности система без модификације класа [12]. За решавање проблема проширења функционалности користи се образац посетилац (енг. *Visitor*). Посетилац припада категорији образаца понашања [11].

Намена посетиоца је раздвајање алгоритма од структуре података над којом се оперише [11]. У језичким процесорима, структура података је представљена чворовима AST-а [12]. Логика за семантичку анализу, оптимизацију или серијализацију локализује се унутар засебних класа, уместо да се уграђује директно у чворове стабла [12].

Посетилац се заснива на механизму двоструког упућивања (енг. *double dispatch*) [11]. Двоструко упућивање зависи истовремено од типа конкретног објекта посетиоца и од типа конкретног елемента структуре [11]. Структура посетиоца (слика 2) обухвата две компоненте:

- апстракцију елемента (енг. *Element*) која дефинише интерфејс са методом *accept(v: Visitor)* за преусмеравање позива конкретних чворова и
- апстракцију посетиоца која дефинише *visit* методе за извршавање алгоритама над конкретним класама елемената [11].



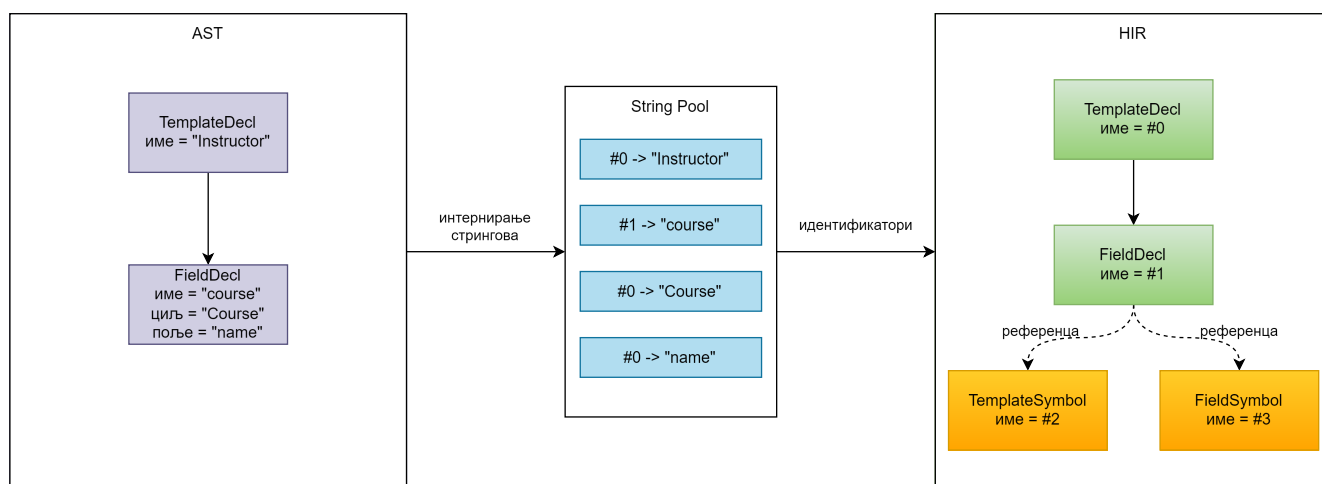
Слика 2: Структура дизајнерског обрасца посетилац [11]

Посетилац се примењује када су испуњени услови да:

- објектна структура садржи елементе различитих класа са специфичним операцијама за сваку конкретну класу,
- постоји потреба за извршавањем несродних операција над елементима без додавања нових метода у класе структуре, те да
- се хијерархија класа објектне структуре ретко мења услед учесталог додавања нових операција над структуром [11].

2.3.4 Високонивојска међурепрезентација

HIR представља корак апстракције у коме се текстуална структура програма трансформише у оријентисани граф релација [10]. Слика 3 приказује разлику између AST-а и HIR-а на примеру једне декларације доменског ентитета. Док AST чува текстуалне идентификаторе и прати синтаксичку структуру програма, HIR користи интерниране идентификаторе и експлицитне везе ка симболима, чиме моделује семантичке односе између конструктора. Изградња HIR-а отпочиње након валидације AST-а. Трансформација се заснива на процесу спуштања (енг. *lowering*). Спуштање елиминише преостале синтаксне декорације и своди изразе на каноничке облике [9].



Слика 3: Поређење AST-а и HIR-а на примеру декларације доменског ентитета.

Улога HIR-а је разрешавање референци, типова и међусобних односа између симбола [10]. HIR интегрише локалне симболе тренутног документа са глобалним симболима који су учитани из модула. Текстуални идентификатори су у фази спуштања замењени интернираним нумеричким кључевима. Фокус се тиме преусмерава са начина на који је код написан на семантичко значење и међусобну повезаност објеката. Компактност релационог графа чини HIR погодним за дуготрајно чување, поновно учитавање модуларних целина и генерисање излазних података.

2.4 Систем модула

Развој језичких система захтева декомпозицију спецификација на мање, независне логичке целине. Концепт модуларности почива на дефинисању изолованих јединица превођења (енг. *compilation units*) које међусобно комуницирају путем прецизно одређених интерфејса [3]. Подршка за модуле унутар преводиоца подразумева стриктно раздвајање локалних опсега видљивости (енг. *scope*) од увезених, спољних симбола. Имплементација система модула намеће потребу за трајним чувањем и дељењем међурезултата превођења на нивоу датотечног система.

Повезивање више изворних датотека у програм доводи до колизије симбола. Колизија настаје када два независна модула дефинишу ентитете са истим текстуалним називом. Проблем колизије се решава увођењем простора имена (енг. *namespace*). Простор имена је изоловани контекст у којем сваки идентификатор мора бити јединствен [6]. Унутар архитектуре преводиоца, имплементација простора имена захтева транзицију са локалног на глобално адресирање [9]. Глобално адресирање се реализује путем композитних кључева који обједињују идентификатор простора имена (модула) и локални идентификатор ентитета [10]. Примена композитних кључева обезбеђује униформну обраду свих симбола у меморији, независно од њиховог физичког порекла.

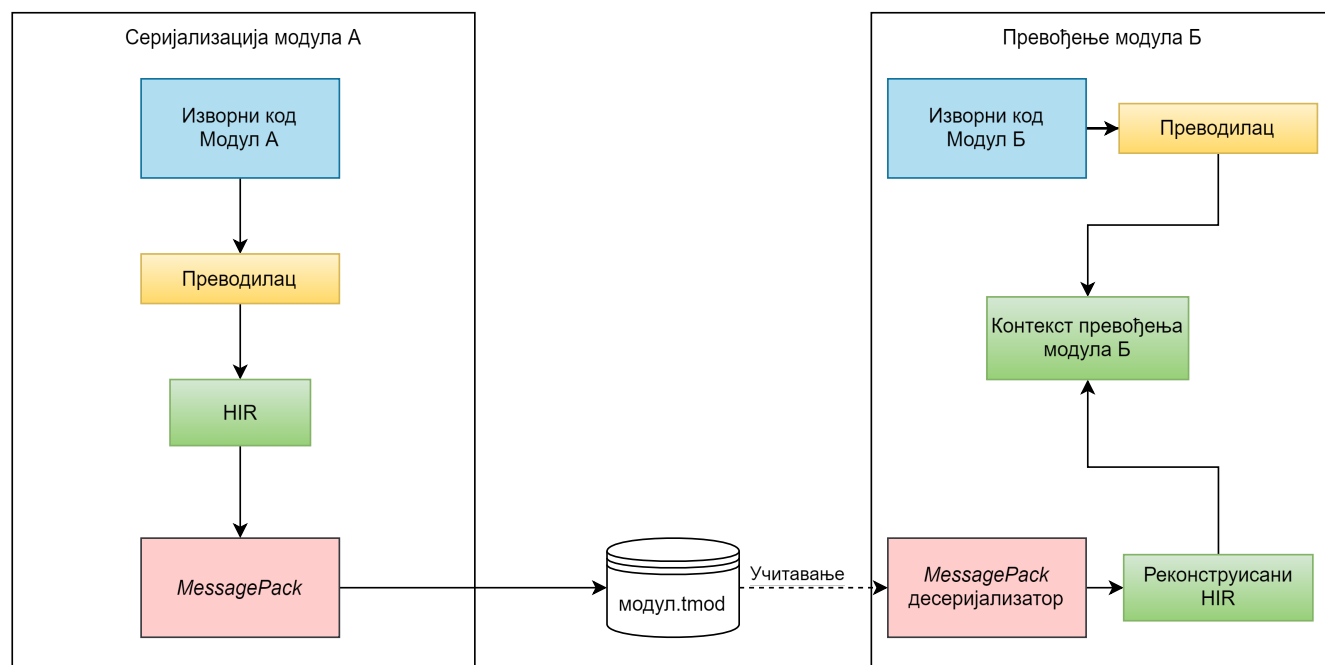
Независно превођење модула резултује стварањем модула који не познају коначни адресни простор програма. Идентификатори унутар модула су релативни и важе искључиво у контексту тог модула [9]. Процес спајања независних модула у извршну целину назива се повезивање (енг. *linking*) [6]. Релокација (енг. *relocation*) је корак током повезивања који ажурира нумеричке референце и меморијске адресе унутар учитаног кода (табела 1) [6]. Учитавање спољног модула захтева транслацију релативних идентификатора у вредности које одговарају тренутном глобалном стању система [10]. Транслација спречава преклапање адреса између увезених библиотека и обезбеђује конзистентност семантичког графа програма.

Табела 1: Пресликавање локалних идентификатора у глобални адресни простор током релокације.

Ентитет	Стање на диску	Стање у меморији након релокације
Структура_А	LocalId(1) унутар зависности	GlobalId(ModuleId(4), LocalId(1))
Структура_Б	LocalId(2) унутар зависности	GlobalId(ModuleId(4), LocalId(2))
Локална променљива	LocalId(1) у тренутном фајлу	GlobalId(ModuleId(0), LocalId(1))

2.4.1 Бинарна серијализација и формати без шеме

Подаци се унутар процеса чувају у меморијским структурама. Трајно складиштење структура или њихов пренос кроз мрежу захтева претварање података у линеарни низ бајтова. Поступак претварања података у линеарни низ бајтова назива се серијализација (енг. *serialization* или *encoding*) [13]. Серијализација IR омогућава креирање независних модула. Процес омогућава имплементацију засебног преводјења (енг. *separate compilation*). Преведени модули се чувају као независне датотеке. Могућност увоза елиминише потребу за поновном синтаксном анализом изворног кода спољних компоненти [3]. Концепт серијализације и увоза модула приказан је на слици 4.



Слика 4: Процес серијализације модула и његовог увоза током преводјења.

Приликом избора механизма за перзистентност IR-а, подела обухвата текстуалне и бинарне формате [13]. Текстуални формати омогућавају читљивост и лако отклањање грешака, али уводе додатни рачунарски трошак (енг. *parsing overhead*) [13]. Трошак настаје због потребе за лексичком и синтаксном анализом при сваком учитавању датотеке. Бинарни формати компактно записују структуре података у облик близак изворном меморијском распореду. Компактно записивање смањује заузеће простора на диску и убрзава учитавање већ преведених модула.

Бинарни формати се деле на системе са строго дефинисаном шемом и самоописујуће формате без унапред одређене шеме (енг. *schemaless*). Системи са шемом (нпр. *Protocol Buffers* и *Apache Avro*) захтевају претходно моделовање података [13]. Пре покретања програма неопходно је извршавање алата за генерисање изворног кода. Фиксна шема пружа максималну компактност јер бинарни запис не садржи називе поља. Крутоост структуре уноси инжењерски трошак уколико се IR често мења током развоја. Самоописујући формати (нпр. *MessagePack*) чувају метаподатке о типовима заједно са вредностима. Складиштење пра-

тећих ознака омогућава брзу еволуцију објектног модела у раним фазама развоја језика. Самоописујућа решења спајају флексибилност текстуалних записа са перформансама бинарног кодирања [13].

Формат *MessagePack* представља бинарну алтернативу JSON запису [14]. Формат задржава флексибилност JSON-а уз смањење просторне комплексности [14]. Ефикасност се постиже коришћењем префикса типа (енг. *type prefix*). Један бајт у систему истовремено дефинише тип податка и његову дужину [14]. Префикс типа елиминише потребу за текстуалним знаковима раздвајања приликом серијализације података.

2.5 Генерисање података

Генерисање тест података на основу декларативног кода заснива се на стохастичком моделовању и рачунарској симулацији [15]. Статичка ограничења из доменског језика преводе се у излазне записе применом алгоритама случајног избора. Алгоритми случајног избора симулирају обрасце података у софтверским системима.

2.5.1 Лења евалуација и токовна обрада података

Лења евалуација (енг. *lazy evaluation*) је стратегија извршавања која одлаже израчунавање израза до тренутка када систем захтева његов резултат [16]. Стриктна евалуација (енг. *eager evaluation*) одмах израчунава све вредности и чува их унутар структура података. Лењи приступ генерише елементе појединачно и на захтев (енг. *on-demand*) [17].

Рачунарске системи реализују лењу евалуацију при генерисању података употребом итератора и токовне обраде (енг. *stream processing*) [16]. Алокација скупа података у оперативној меморији уноси просторну сложеност $\mathcal{O}(N)$ у односу на број записа N . Токовна обрада елиминише меморијско преоптерећење. Систем генерише појединачни запис, уписује вредност на излазни ток и ослобађа простор за наредну итерацију [13]. Механизам редукује просторну сложеност на константни ниво $\mathcal{O}(1)$. Константна просторна сложеност одржава стабилну меморијску потрошњу независно од укупног броја излазних записа.

2.5.2 Псеудослучајни генератори

Симулација у рачунарским системима ослања се на псеудослучајне генераторе бројева (енг. *Pseudo-Random Number Generator*, PRNG) [18]. Рад алгорита отпочиње прихватањем почетне вредности под називом семе (енг. *seed*). Генератор користи семе за креирање низа бројева кроз сукцесивне математичке трансформације [19]. Математичке трансформације се најчешће заснивају на модуларној аритметици или манипулацији битовима [19]. Резултујући низ је детерминистички условљен почетном вредношћу. Излазни подаци пролазе статистичке тестове независности [18]. Примењени модел симулира својства случајности [18]. Избор PRNG алгорита утиче на укупне перформансе система. Алгоритам одређује брзину извршавања рачунарских симулација и статистички квалитет генерисаних тест примера.

Репродуктивност представља особину система за генерисање података која подразумева да идентична улазна спецификација доводи до конзистентних излазних резултата [15]. Фиксирање почетне вредности омогућава репликацију интерног стања генератора. Поновљивост обезбеђује детерминистичку репродукцију грешака откривених током извршавања тестова. Стабилност излазних структура оптимизује радне процесе унутар система за аутоматизовану интеграцију. Једнообразност података олакшава размену тест сценарија међу члановима развојног тима.

2.5.3 Статистичке расподеле

Униформна расподела је статистичка расподела у којој сви исходи унутар дефинисаног интервала $[a, b]$ имају једнаку вероватноћу избора [15]. Функција густине расподеле за непрекидни случај дефинисана је као:

$$\mathcal{U}(x) = \frac{1}{b - a}$$

за $a \leq x \leq b$.

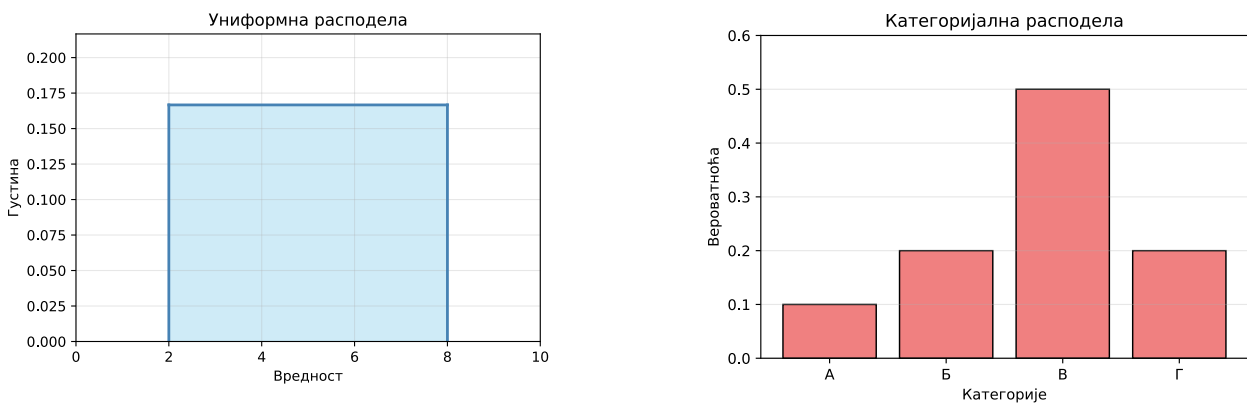
У контексту система за генерисање података, када корисник дефинише нумерички или текстуални опсег (нпр. распон година или интервал датума) без навођења додатних метаподатака о структури, систем примењује дискретну или непрекидну униформну расподелу [19]. Униформна расподела додељује једнаку вероватноћу свим вредностима унутар дефинисаног опсега.

Тежински избор (енг. *weighted selection*) је рачунарска имплементација категоријалне расподеле. Сваки потенцијални исход x_i поседује дефинисан фактор тежине w_i који одређује његову важност [15]. Вероватноћа избора појединачног исхода p_i израчунава се нормализацијом његове тежине у односу на суму свих тежина у скупу:

$$p_i = \frac{w_i}{\sum_{j=1}^n w_j}$$

Алгоритамски, тежински избор се реализује конструисањем низа кумулативних сума [19]. Систем генерише случајан број U из униформне расподеле у интервалу $[0, \sum w_j)$, а затим претрагом проналази индекс сегмента коме тај број припада. Временска комплексност алгоритма зависи од одабраног алгоритма претраживања. Секвенцијално претраживање низа уноси линеарну временску сложеност реда $\mathcal{O}(N)$, где вредност N представља укупан број категорија. Примена бинарне претраге оптимизује алгоритам и редукује комплексност на $\mathcal{O}(\log N)$. Тежински избор обезбеђује моделовање система у којима поједини ентитети имају унапред одређену, већу учесталост појављивања.

Теоријски приказ функције густине униформне расподеле и расподеле вероватноћа категоријалне расподеле дат је на слици 5. Са леве стране приказана је константна густина униформне расподеле на интервалу $[a, b]$, док су са десне стране приказане вероватноће избора појединачних категорија, одређене њиховим тежинама.



Слика 5: Теоријски приказ униформне (лево) и категоријалне расподеле (десно).

2.5.4 Референцијални интегритет и графови зависности

Генерисање скупова података који садрже међусобне везе захтева очување референцијалног интегритета [20]. У контексту синтетизације података, референцијални интегритет је структурно ограничење у коме вредност једног атрибута (страног кључа) мора постојати унутар скупа вредности референцираног ентитета [20]. Ограничење референцијалног интегритета онемогућава независно генерисање вредности и уводи релациону међузависност унутар система.

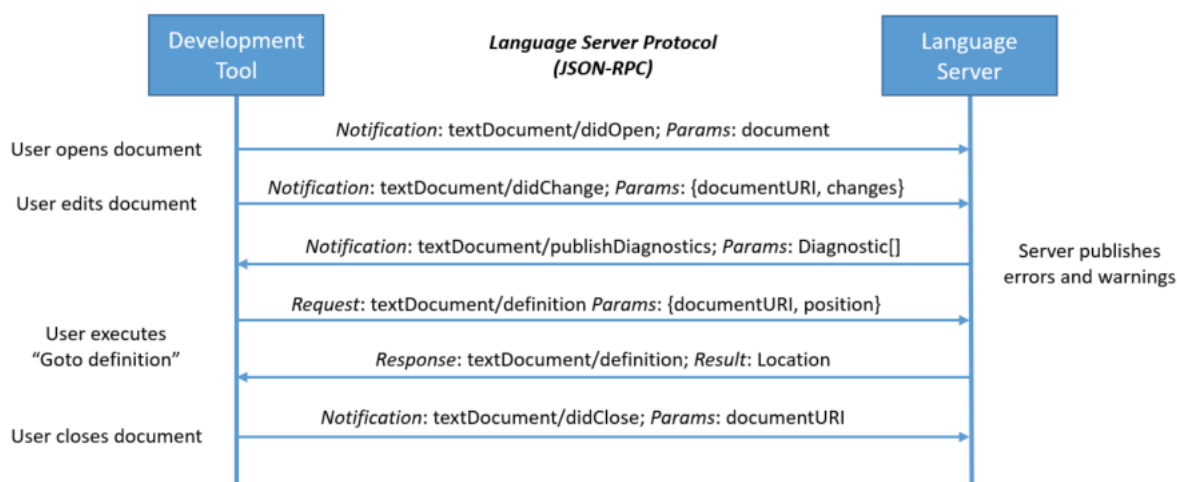
Међусобне условљености се моделују помоћу усмерених графова зависности (енг. *dependency graph*) [6]. Чворови графа представљају ентитете, док усмерене гране означавају постојање референцијалних веза између њих. Проналажење валидног редоследа извршавања генерације захтева примену алгоритма за тополошко сортирање над конструисаним графом [6]. Тополошко сортирање поставља чворове графа у линеаран низ у коме сваки референцирани ентитет претходи зависном ентитету. Употреба тополошког

сортирања успоставља редослед којим се сви примарни објекти креирају у меморији пре отпочињања обраде зависних поља.

2.6 Протокол језичких сервера

LSP је стандардни протокол комуникације између развојних окружења и језичких сервера [2]. Стандард раздваја језички неутралан уредник текста (енг. *editor*) од језичког сервера [4]. Језички сервер извршава статичку и семантичку анализу без познавања специфичности развојног окружења [4]. Развојно окружење управља корисничким интерфејсом без познавања дубље семантике програмског језика [4].

Уредник текста и језички сервер комуницирају путем *JavaScript Object Notation Remote Procedure Call* (JSON-RPC) протокола [2]. Током уређивања, уредник текста шаље садржај документа и корисничке захтеве, док језички сервер враћа резултате анализе и податке потребне за функционалности уредника текста [5]. Слика 6 приказује ток комуникације.



Слика 6: Комуникација између уредника текста и језичког сервера током уређивања документа [2].

Развој подршке за развојна окружења захтевао је засебну имплементацију за сваки програмски језик [4]. Приступ доводи до укупног броја имплементација који одговара производу броја језика и броја развојних окружења [21]. Последице децентрализоване архитектуре обухватају високе трошкове развоја и спорију испоруку софтверских алата [5]. Различита развојна окружења често показују неконзистентно понашање приликом анализе истог изворног кода [5]. Комплексност одржавања онемогућава развојним тимовима пружање квалитетне подршке за више уредника текста [4].

Архитектура користи клијент-сервер модел за раздвајање логике уредника текста од семантике језика [2]. Развојно окружење преузима улогу клијента који управља корисничким интерфејсом [4]. Језички сервер ради као независан процес задужен за статичку анализу кода [21]. Комуникација се ослања на стандардизоване поруке унутар JSON-RPC протокола [2]. Протокол дефинише одређен животни циклус сесије кроз захтеве за иницијализацију и гашење сервера [2]. Размена података обухвата асинхроно слање обавештења о изменама унутар отворених датотека [2]. Сервер извршава анализу у позадини и самостално шаље дијагностичке поруке клијенту [2].

Протокол стандардизује функционалности за подршку током кодирања [2]. Основна могућност обухвата аутоматско допуњавање кода на основу тренутног контекста [2]. Систем омогућава прелазак на место дефиниције симбола у изворном коду [2]. Функционалност претраге референци проналази сва појављивања изабраног идентификатора у пројекту [2]. Приказ информација кроз лебдећи прозор пружа увид у типове података и документацију симбола [2]. Генерисање дијагностичких порука приказује синтаксне и семантичке грешке у реалном времену [21]. Подршка за рефакторисање укључује сигурно аутоматизовано преименовање симбола кроз све датотеке [2].

Унутар LSP-а, перформансе и одзивност корисничког интерфејса зависе од асинхроне обраде захтева [2]. Корисник непрекидно мења изворни код брзим уносом карактера, процеси семантичке анализе могу постати застарели пре него што се изврше до краја. Да би се спречило непотребно заузеће процесорских ресурса, LSP уводи механизам отказивања задатака путем токена за отказивање (енг. *cancellation token*) [2]. Клијент шаље обавештење о отказивању за сваки захтев чији је резултат постао сувишан услед новог стања у уреднику текста. Језички сервер периодично проверава статус токена током дуготрајних операција, прекида тренутно извршавање у случају активације и ослобађа ресурсе за наредну итерацију анализе.

2.7 Архитектура командних интерфејса и рутирање захтева

Развојни алати захтевају механизме за комуникацију са корисником. Софтверска архитектура улазних тачака најчешће користи командни интерфејс (енг. *Command Line Interface*, CLI) [22]. Системи примењују обрасце за централизовано управљање захтевима [23].

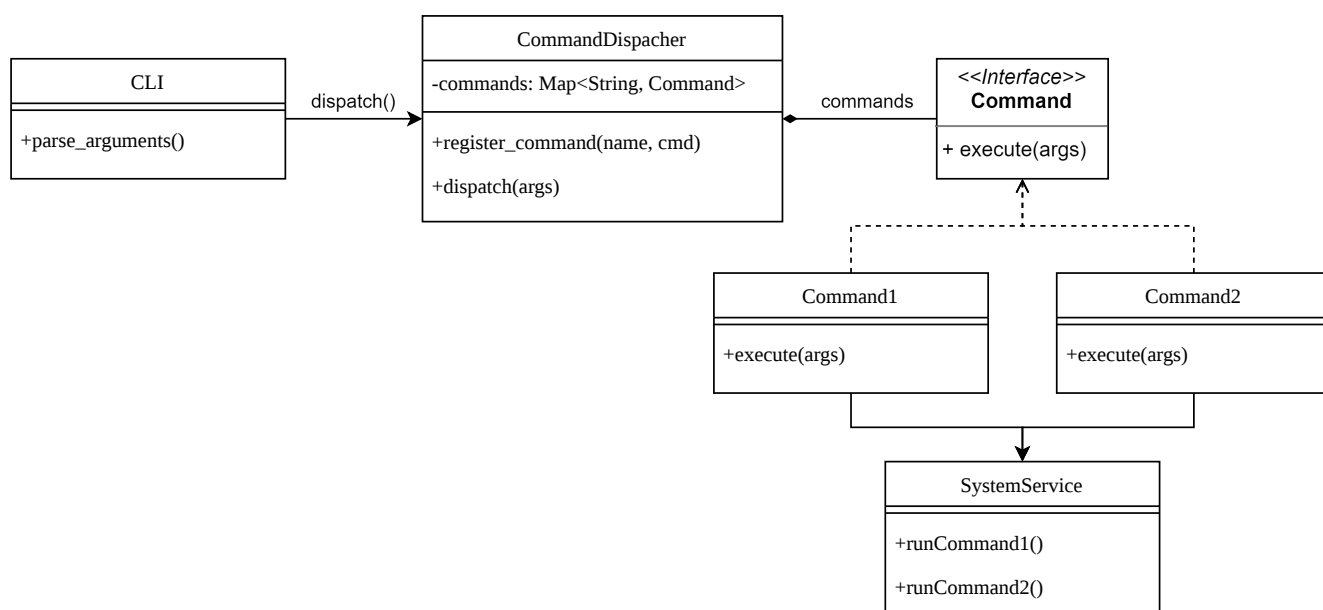
CLI је текстуални механизам за комуникацију са апликацијом. Омогућава извршавање програма кроз стандардне токове улаза и излаза. Стандардни токови омогућавају преусмеравање и уланчавање података између различитих програма (енг. *piping*) [22].

CLI служи као улазна тачка за преводиоце доменских језика [3]. Текстуални интерфејс омогућава аутоматизацију и повезивање са системима за континуирану интеграцију. Уместо ручне обраде текста, развојни екосистеми пружају специјализоване библиотеке које превode текстуалне аргументе у структурисане податке. Употреба специјализованих библиотека обезбеђује стриктно раздвајање фазе обраде параметара од оперативне логике самог програма.

2.7.1 Дизајнерски образац диспечера команди

Након обраде улазних аргумената, систем рутира захтеве ка одговарајућим компонентама. Принцип јединствене одговорности (енг. *Single Responsibility Principle*) налаже раздвајање обраде улаза од доменске логике [24]. Синтаксна анализа аргумената и извршавање оперативне логике представљају одвојене функционалне целине.

Образац диспечера команди обезбеђује структурисано рутирање корисничких захтева. Архитектура диспечера комбинује принципе обрасца команда (енг. *Command pattern*) [11] и процесора команди (енг. *Command Processor*) [25]. Док образац команда енкапсулира кориснички захтев у независни објекат, диспечер преузима улогу централног контролера који на основу улазних параметара активира одговарајућу команду. Структура дизајнерског обрасца диспечера команди приказана је на слици 7.



Слика 7: Структура дизајнерског обрасца диспечер команди.

Примена диспечера команди обезбеђује поштовање отворено-затвореног принципа (енг. *Open/Closed principle*) [24]. Додавање нове функционалности захтева креирање нове класе команде. Структура диспечера и постојећи сервиси остају непромењени приликом проширења.

Пројектовање и имплементација алата за генерисање података заснованих на наменским језицима захтевају дефинисање тренутног технолошког стања и сродних софтверских архитектура. Развој решења обухвата три области: инжењерство програмских језика, системе за дефинисање формалних шема података и платформе за стохастичку синтетизацију тест примера.

Унутар поглавља се приказује преглед постојећих алата унутар наведених категорија. Анализа идентификује предности и ограничења решења, са фокусом на начине разрешавања референцијалног интегритета, експресивности синтаксе и интеграције са уредницима текста. Упоредни приказ служи као основа за евалуацију која оправдава развој новог доменског језика и пратеће архитектуре преводиоца.

3.1 Језици специфични за домен

Развој DSL-а најчешће се не изводи изградњом преводиоца од почетног нивоа, већ применом специјализованих мета-језичких алата и развојних окружења који се називају језичке радионице (енг. *language workbench*). Системи омогућавају декларативно дефинисање граматике, семантичких правила и пратећих компоненти за уређивање кода [3]. Анализирани алати показују разлике у погледу комплексности архитектуре и интеграције са екстерним системима.

3.1.1 *textX*

Алат *textX* је мета-језик имплементиран у програмском језику *Python*. Наменен је за изградњу DSL-а [26]. Систем се заснива на синтаксном анализатору *Arpeggio*. *Arpeggio* користи концепт граматике анализе синтаксних израза (енг. *Parsing Expression Grammar*) [27]. На основу текстуалне граматике, *textX* конструише мета-модел у меморији и извршава валидацију улазних датотека.

Предности алата су брзо прототипизовање и интеграција са *Python* екосистемом. Корисник дефинише језичка правила кроз синтаксу сличну проширеној *Backus-Naur-овој* форми (листинг 1). Систем мапира препознате синтаксне структуре у динамичке објекте. Мапирање смањује когнитивно оптерећење инжењера, јер развој не захтева ручно писање класа за AST.

```
Model: entities+=Entity;
Entity: 'entity' name=ID '{' properties+=Property '}';
Property: name=ID ':' type=ID;
```

Листинг 1: Дефиниција граматике у алату *textX*

Архитектура алата *textX* показује ограничења у домену стохастичке синтезе података. Систем не поседује изворне семантичке операторе за дефинисање тежинских вероватноћа унутар граматике. Алгоритам за разрешавање референци унутар алата *textX* ограничен је на повезивање објеката. Граф зависности и валидација референцијалног интегритета се имплементирају кроз *Python* код након фазе синтаксне анализе. Постоји подршка за комуникацију са развојним окружењима путем пројекта *textX-LS*. Подршка захтева инсталацију и конфигурацију додатних екстерних библиотека. Језички сервер није изворно интегрисан у језгро самог преводиоца као јединствен процес.

3.1.2 *JetBrains MPS*

JetBrains MPS је језичка радионица заснована на концепту пројекционог уређивања кода [28]. Систем не користи синтаксни анализатор за претварање текстуалног записа у AST. Корисник модификује структуру стабла у меморији. Измена кода се извршава кроз наменске визуелне пројекције унутар окружења.

Предност *JetBrains MPS* је подршка за комбиновање различитих нотација унутар истог модела [29]. Код се може приказати у облику текста, табела, математичких формула или графичких дијаграма. Непостојање фазе синтаксне анализе елиминиса појаву синтаксних двосмислености током дефинисања језика. Окружење омогућава поновну употребљивост развијених језичких компоненти.

Архитектура *JetBrains MPS* показује ограничења у погледу интеграције и намене за синтезу података. Систем захтева коришћење монолитног развојног окружења. Захтев онемогућава једноставну интеграцију са спољним текстуалним уредницима путем LSP протокола. Пројекциони приступ уводи когнитивно оптерећење при уносу кода због одсутности текстуалне синтаксе [29]. Систем не поседује изворне семантичке операторе за дефинисање стохастичких вероватноћа. Разрешавање комплексних релационих зависности између ентитета захтева ручно писање трансформационих генератора у језику *BaseLanguage* [28].

3.2 Системи за дефинисање шема

Системи за дефинисање структура и шема података примарно су намењени валидацији, серијализацији и размени информација унутар дистрибуираних софтверских архитектура [13]. Иако изворна намена није стохастичко генерисање података, технологије поседују формалне описе ентитета и релација. Користе се као полазна основа за аутоматизовано креирање тест сценарија, при чему се уочавају ограничења услед недостатка изворних стохастичких оператора унутар мета-модела.

3.2.1 JSON Schema

JSON Schema је формат за декларативно дефинисање и валидацију структура унутар JSON записа [30]. Систем омогућава постављање структуралних ограничења над подацима унутар дистрибуираних софтверских архитектура [13] (листинг 2). Технологија служи за верификацију исправности размене информација између независних апликација.

JSON Schema подржава поновну употребљивост компоненти помоћу механизма синтаксних показивача [30]. Употреба оператора *\$ref* омогућава експлицитно повезивање локалних и удаљених објеката унутар дефинисане шеме. Провера усклађености записа изводи се аутоматски преко софтверских библиотека доступних у GPL.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Course",
  "type": "object",
  "properties": {
    "course_id": { "type": "integer" },
    "name": { "type": "string", "pattern": "^Kurs-[0-9]{3}$" },
    "department": { "type": "string", "enum": ["ComputerScience", "Mathematics"] }
  },
  "required": ["course_id", "name", "department"]
}
```

Листинг 2: Пример *JSON Schema*

Примена формата у домену стохастичке синтезе података показује недостатке. Мета-модел не поседује изворне семантичке операторе за дефинисање тежинских вероватноћа избора вредности [30]. Аутоматизовано креирање тест сценарија на основу *JSON Schema* захтева имплементацију нестандартних екстензија изван званичне спецификације.

3.2.2 Protocol Buffers

Protocol Buffers је језик за дефинисање интерфејса (енг. *Interface description language*) и формат за бинарну серијализацију података [31]. Технологија омогућава размену информација унутар микросервисних архитектура [13]. Спецификација модела се дефинише декларативно унутар изворних текстуалних датотека са екстензијом *.proto* (листинг 3).

Предности *Protocol Buffers* су перформансе и компактност генерисаног бинарног записа [13]. Наменски преводац *protoc* преводи дефинисане поруке у структуре података за различите програмске језике [31]. Формат подржава унапредиву компатибилност шема кроз систем нумеричких ознака поља.

```
syntax = "proto3";

message Korisnik {
    int32 korisnik_id = 1;
    string ime = 2;
}
```

Листинг 3: Шема поруке унутар *proto* датотеке

Архитектура система *Protocol Buffers* показује ограничења у контексту генерисања тест података. Мета-модел нема изворне семантичке операторе за конфигурисање стохастичких вероватноћа унутар шеме [31]. Систем не поседује механизме за аутоматско препознавање и очување референцијалног интегритета између засебних порука. Разрешавање комплексних релација између ентитета захтева имплементацију додатних алгоритама на нивоу корисничке апликације.

3.3 Системи за генерисање тест података

Системи за генерисање тест података чине категорију софтверских решења за аутоматизацију тестирања. Развојни фокус је синтеза реалистичних вредности унутар база података и корисничких интерфејса. Употреба наменских генератора смањује ручни напор приликом креирања сложених тест сценарија.

3.3.1 *Faker*

Faker је програмска библиотека доступна у већини GPL, намењена за генерисање псеудослучајних података [32]. Интеграција у софтверске пројекте изводи се кроз изворни код тест оквира (листинг 4).

Предност *Faker* библиотеке је флексибилност унутар развојног окружења. Корисник комбинује генерисане вредности са пословном логиком апликације. Локализовани провајдери омогућавају прилагођавање излазних стрингова језичким и културолошким форматима.

```
from faker import Faker
generator = Faker()

print(generator.name())
print(generator.email())
```

Листинг 4: Имплементација псеудослучајног генератора у језику *Python*

Архитектура библиотеке *Faker* уводи императивни трошак при моделирању релационих структура [32]. Развој захтева ручну конструкцију меморијских бафера за чување примарних кључева ради очувања референцијалног интегритета. Систем захтева експлицитно дефинисање редоследа извршавања петљи за сваки повезани ентитет. Алат не поседује изворне семантичке операторе за декларативно конфигурисање стохастичких тежина на нивоу модела података.

3.3.2 *Mockaroo*

Mockaroo је веб-платформа заснована на GUI за генерисање реалистичних тест података (слика 8) [33]. Систем омогућава конфигурисање излазних датотека у форматима као што су CSV, JSON и SQL [33]. Платформа поседује уграђену базу генератора за различите индустријске и пословне домене [33].

Предност платформе је интуитивно дефинисање међусобних релација између ентитета кроз визуелне компоненте [33]. Систем подржава механизме за очување референцијалног интегритета помоћу унакрсног повезивања колона [33]. Корисници примењују наменске формуле за увођење стохастичких тежина и вероватноћа при синтези вредности [33].

The screenshot displays the Mockaroo web interface for configuring data generation. It features a table with columns for 'Field Name', 'Type', and 'Options'. Six fields are listed: 'id' (Row Number), 'first_name' (First Name), 'last_name' (Last Name), 'email' (Email Address), 'gender' (Gender), and 'ip_address' (IP Address v4). Each field has a 'blank' percentage set to 0%, a 'Σ' icon, and an 'X' icon. Below the table are buttons for '+ ADD ANOTHER FIELD' and 'GENERATE FIELDS USING AI...'. At the bottom, there are settings for '# Rows' (1000), 'Format' (CSV), 'Line Ending' (Unix (LF)), and 'Include' (checked for 'header', unchecked for 'BOM').

Слика 8: GUI платформе *Mockaroo* за конфигурацију генератора података

Архитектура алата *Mockaroo* показује ограничења у погледу интеграције и локалне контроле. Алат функционише као затворена инфраструктура, што онемогућава локално извршавање унутар изолованих мрежа [33]. Одсуство текстуалне синтаксе отежава верзионисање и праћење измена модела кроз системе за контролу верзија [3]. Бесплатна верзија платформе ограничава максималан број генерисаних редова по датотеци [33].

3.4 Закључна разматрања и упоредни приказ

Евалуација технологија открива компромис између изражајности мета-модела и когнитивног оптерећења развоја [3]. Текстуални формати омогућавају интеграцију са системима за контролу верзија кода [21]. Графичке платформе убрзавају визуелну конфигурацију релација уз ограничење локалног извршавања софтвера [3]. Бинарни формати оптимизују мрежни пренос података уз губитак читљивости изворног записа [13]. Табела 2 садржи таксономију карактеристика, предности и ограничења свих анализираних решења.

Табела 2: Поређење предложеног решења са постојећим алатима

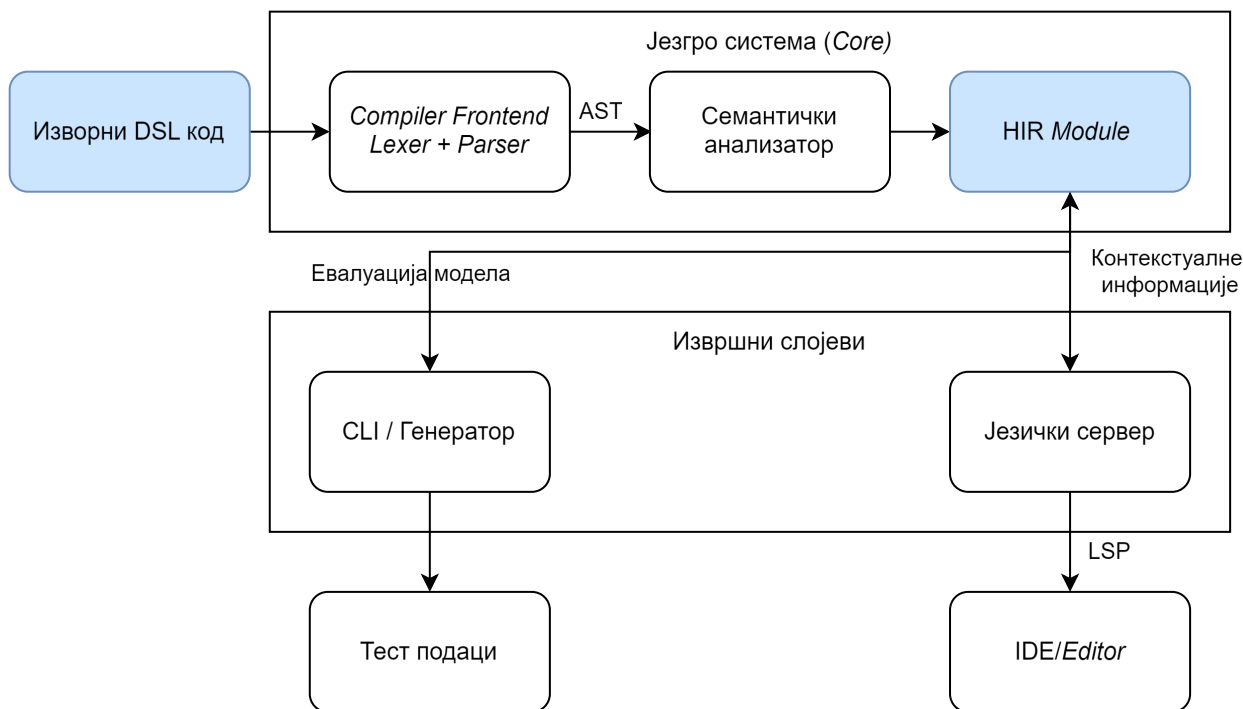
Алат / Систем	Приступ моделовању	Референцијални интегритет	Стохастички оператори	Изворни LSP
<i>textX</i>	Текстуални DSL	Делимично	Не	Да (екстерно)
<i>JetBrains MPS</i>	Пројекциони DSL	Да	Не	Не
<i>JSON Schema</i>	Шема (JSON)	Да	Не	Делимично
<i>Protocol Buffers</i>	Шема (бинарна)	Не	Не	Не
<i>Faker</i>	Програмски API	Ручно	Не	Не примењује се
<i>Mockaroo</i>	GUI	Да	Да	Не примењује се
Предложено решење	Текстуални DSL	Аутоматски (граф)	Да	Да

Циљ решења је развој DSL-а који омогућава генерисање тест података. Ово поглавље описује целокупан процес дизајна система. Систем као улаз прима текст изворне датотеке и захтеве уредника текста. Захтеви се шаљу путем LSP-а. Излаз система чине текстуалне датотеке са генерисаним тест подацима и одговори формирану у складу са LSP спецификацијом.

Редослед излагања прати логичку структуру система. Архитектура система (поглавље 4.1) приказана је на највишем нивоу апстракције. Дизајн језика (поглавље 4.2) дефинише синтаксу и структуралне елементе. Семантичка анализа (поглавље 4.3) описује механизме валидације изворног кода. HIR (поглавље 4.4) представља модел података који премोшћује јаз између текстуалног записа и извршних слојева. Дизајн модула (поглавље 4.5) објашњава организацију независних компоненти. Дизајн генератора података (поглавље 4.6) дефинише методе за стохастичко узорковање. Напоследку, дизајн језичког сервера (поглавље 4.7) описује управљање захтевима развојног окружења, након чега су изложени проблеми настали током дизајнирања.

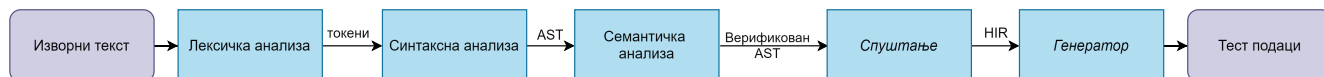
4.1 Архитектура система

Архитектура система пројектована је по принципу дељеног језгра. Језгро система обухвата лексичку, синтаксну и семантичку анализу. Раздвајање језгра од извршних слојева омогућава поновну употребљивост кода. Глобална структура софтвера и међусобне везе између главних компоненти приказане су на слици 9.



Слика 9: Дијаграм архитектуре система

Превођење изворног кода у финалну структуру организовано је као ланац обраде (енг. *pipeline*). Ланац обраде постепено трансформише податке. Ток података приказан је на слици 10.



Слика 10: Ланац обраде у преводиоцу.

4.2 Дизајн језика специфичног за домен

Дизајн језика се заснива на декларативном принципу. Корисник описује структуру и ограничења података. Поступак генерисања вредности је апстрахован. Језик је пројектован са статичким системом типова. Статичка типизација омогућава рано откривање грешака у изворном коду путем језичког сервера [1], [3].

4.2.1 Директиве и конфигурација контекста

Синтаксне директиве почињу симболом `@`. Директиве служе за глобалну конфигурацију окружења, управљање модулима и дефинисање метаподатака преводилаца. Пример директива приказан је на листингу 5. Употреба директива обезбеђује подешавање извршног окружења без измене логике самих шаблона.

```
@import skalarne_definicije;
@output json { pretty = true; indent = 4; }
@output_path "./izlaz/korisnici.json";
@generate Korisnik[100];
```

Листинг 5: Пример глобалних директива и увоза модула

Директива `@import` омогућава декомпозицију изводног кода на мање модуларне јединице. Излазни формат за размену података и пратећу конфигурацију дефинише директива `@output`. Циљну локацију датотеке на диску одређује директива `@output_path`. Процес генерисања података над изабраним шаблоном покреће извршна директива `@generate`.

4.2.2 Кориснички типови и синтаксна ограничења

Систем типова подржава проширење примитивних типова. Проширење се врши увођењем корисничких типова са специфичним ограничењима. Дефинисање типова са ограничењима приказано је на листингу 6. Кориснички типови омогућавају контролу над доменом генерисаних вредности на нивоу типа података.

```
type GodinaStudija = int [range=1..=4];
type ProsečnaOcena = float [range=6.0..=10.0];
type BrojIndeksa = string_pattern "2026/${#[4]}";
```

Листинг 6: Дефинисање типова са ограничењима.

Синтаксна конструкција `range` ограничава домен дозвољених нумеричких вредности. Генератор примењује кориснички дефинисану униформну расподелу вредности унутар задатог интервала. Кључна реч `string_pattern` користи текстуалне макрое за форматирање псеудослучајних низова карактера.

4.2.3 Колекцијски типови и вишедимензионалне структуре

Систем типова подржава дефинисање колекција кроз типске листе. Листе омогућавају груписање више елемената истог базног типа. Број генерисаних елемената контролише се синтаксним ограничењем `count`. Ограничење `count` дефинише минималан и максималан број чланова колекције. Дефинисање колекцијских типова приказано је на листингу 7.

```
type Grade = int [range=6..=10];
type IntList = [int][count=1..=5];
type IntMatrix = [IntList][count=1..=5];

template Student {
    grades = [Grade][count=1..5];
    tensor = [IntMatrix][count=1..=5];
}
```

Листинг 7: Дефинисање колекцијских типова и вишедимензионалних структура

Конструкт листе прихвата примитивне и кориснички дефинисане типове. Типски систем дозвољава вишеструко улагање листа. Улагање омогућава креирање вишедимензионалних структура попут матрица и тензора. Генератор стохастички одређује коначну дужину сваке листе унутар дефинисаног интервала.

4.2.4 Пробабилистичке енумерације

Енумерације дефинишу коначан скуп вредности и припадајуће тежинске факторе расподеле. Тежински фактори се наводе помоћу оператора `=>`. Уколико тежина није експлицитно наведена, подразумевана вредности је 1. Пример енумерације са факторима тежине приказан је на листингу 8.

```
enum StatusStudenta {
    Budzet          => 7;
    Samofinansiranje => 3;
}
```

Листинг 8: Енумерација са факторима тежине расподеле

Генератор користи дефинисане тежине за стохастичко узорковање вредности. Вероватноћа избора одређене ставке је пропорционална њеном тежинском фактору у односу на збир свих фактора. Пробабилистичке енумерације омогућавају подешавање статистичког профила излазних података.

4.2.5 Структурна композиција и наслеђивање

Моделовање објеката изведено је кроз концепте структура (енг. *struct*) и шаблона (енг. *template*). Структура дефинише логички контејнер за композицију поља. Шаблон представља активну семантичку јединицу над којом се извршава генерација. Наслеђивање поља дефинише се оператором `::`. Структурна композиција и механизам наслеђивања приказани су на листингу 9.

```
struct Lokacija {
    grad    = string;
    drzava  = string;
}
template Osoba {
    ime     = string;
    adresa  = Lokacija;
    id      = string_pattern "0-#{#[5]}";
}
template Zaposleni : Osoba {
    override id = string_pattern "Z-#{#[5]}";
    plata      = int [range=50000..=150000];
}
```

Листинг 9: Структурна композиција и механизам наслеђивања

Шаблон наслеђује сва поља родитељског шаблона. Наслеђивање обухвата и све родитељске структуре. Конфликт истоимених поља кроз хијерархију захтева експлицитно разрешавање. Дизајн језика уводи кључну реч *override* за преклапање родитељских поља. Систем приликом преклапања користи вредност поља које је најближе детету. Изостанак експлицитне декларације за поновљена поља генерише семантичко упозорење. Механизам наслеђивања смањује редундантност приликом спецификације домена. Композиција омогућава хијерархијску организацију података унутар резултујућег записа.

4.2.6 Очување релационог интегритета

Очување логичких веза између генерисаних ентитета реализовано је увођењем кључне речи *ref*. Референцирање вредности ради очувања интегритета приказано је на листингу 10.

```
template Smer {
    sifra = string_pattern "${A[3]}";
}
template Student {
    id      = int;
    smer_kod = ref Smer.sifra;
}
```

Листинг 10: Референцирање вредности ради очувања интегритета

Кључна реч *ref* сигнализира генератору да вредност поља мора бити преузета из скупа већ изгенерисаних вредности. Очување релационог интегритета симулира механизам страних кључева из релационих база података. Страни кључеви спречавају појаву неконзистентних података.

4.3 Дизајн семантичке анализе

Семантичка анализа је средишњи део аналитичког слоја унутар архитектуре система. Синтаксни анализатор прослеђује изграђен AST семантичком слоју. Семантички анализатор гради табелу симбола, врши проверу типова и референци. Излаз семантичке анализе је попуњена табела симбола и скуп дијагностичких порука.

Пример рада семантичког анализатора приказан је на слици 11. Анализатор из изворног кода формира табелу симбола и спроводи семантичке провере. Резултати семантичке провере су дијагностичке поруке у случају нарушавања правила језика.



Слика 11: Пример рада семантичког анализатора.

Изградња табеле симбола је почетна фаза семантичке анализе. Компонента пролази кроз структуру стабла помоћу дизајнерског обрасца посетилац [11]. Модул региструје све дефинисане шаблоне, структуре, еnumерације и корисничке типове. Компонента прати контекст извршавања кроз улазак у опсеге видљивости. Систем детектује невалидне контексте и дуплиране идентификаторе унутар истог опсега.

Провера типова је друга фаза семантичке анализе. Модул за проверу анализира исправност израза и додељених вредности. Компонента утврђује конкретан тип за сваки конструкт у коду. Систем захтева да тежински фактори унутар еnumерација буду целобројне вредности. Компонента контролише број генерисаних ентитета у извршним директивама. Одступање од правила резултује генерисањем семантичке грешке.

Провера референци је завршна фаза семантичке анализе. Компонента контролише постојање свих коришћених идентификатора. Компонента проверава исправност кључне речи *ref* приликом повезивања поља између различитих шаблона. Систем претражује локалну табелу симбола и табеле симбола из увезених модула. Компонента детектује непостојеће родитељске шаблоне у процесу наслеђивања. Провера референци спречава позивање непостојећих поља унутар структура.

4.4 Дизајн високонивојске међурепрезентације

Процес преводјења AST-а у HIR изводи се кроз фазу спуштања [10]. Компонента за спуштање уклања синтаксно-декоративне елементе. Структуре се трансформишу у облике погодне за генерацију података и рад језичког сервера. Процес трансформације користи изоловани контекст који омогућава једнопролазну регистрацију свих дефиниција. Регистрованим дефиницијама се потврђује постојање пре доделе јединствених идентификатора.

HIR обухвата интернирање стрингова, адресирање ентитета и издвајање ограничења. Базен стрингова преводи текстуалне идентификаторе у нумеричке кључеве [10]. Поређење стрингова се своди на операције над целим бројевима. Архитектура система се ослања на концепт униформног глобалног адресирања. Ентитети се идентификују композитним кључем који спаја идентификатор модула и локални идентификатор ставке. Семантички граф програма постаје независан од оригиналног текстуалног записа.

Резултат процеса спуштања је структура модула. Структура концептуално повезује метаподатке, базен стрингова и увозне зависности. Модул се посматра као јединствена и самодовољна целина погодна за пренос између фаза превожња.

4.5 Дизајн модула

Дизајн модуларног система инспирисан је архитектуром изолованих метаподатака преводиоца за програмски језик *Rust* [10]. Архитектура омогућава организацију дефиниција кроз више датотека. Систем подржава симултани рад језичког сервера са више табела симбола. Систем почива на четири дизајнерске компоненте:

- међумодуларно референцирање,
- релокација модула и разрешавање зависности,
- перзистентност и серијализација и
- реконструкција стања.

Пример увоза спољашњег модула и референцирања његових симбола приказан је на листингу 11.

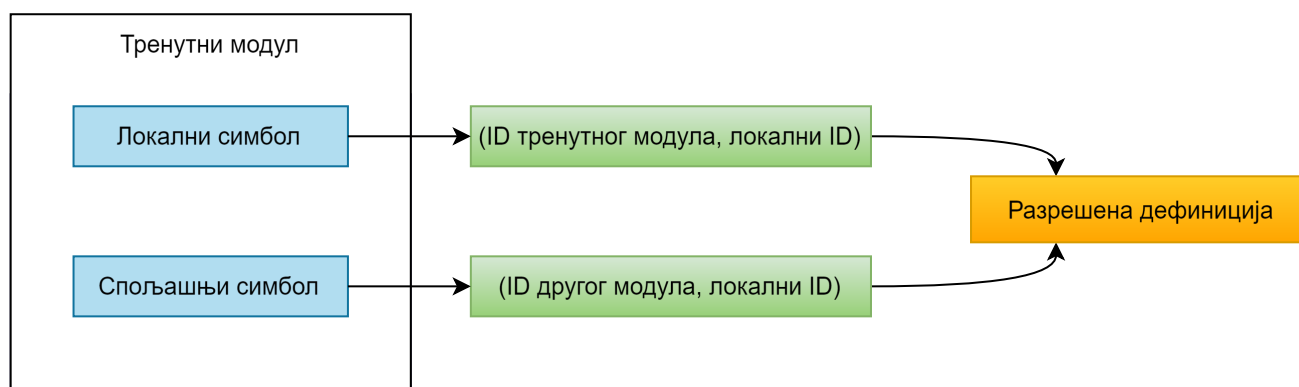
```
// Датотека 1: zajednicki_tipovi.testa
struct KontaktPodaci {
    email    = string_pattern "${a[8]}@kompanija.com";
    telefon  = string_pattern "+3816${#[8]}";
}

// Датотека 2: glavni.testa
@import zajednicki_tipovi;
template Klijent {
    kontakt = KontaktPodaci; // Референцирање спољног модула
}
```

Листинг 11: Увоз модула и референцирање споља дефинисаног типа

4.5.1 Међумодуларно референцирање и релокација

Повезивање симбола између различитих модула користи механизам композитних кључева. Дизајн елиминисе потребу за засебним структурама података при раду са локалним и спољашњим симболима. Апстрахује се физичка локација дефиниције ентитета. Разрешавање локалних ставки захтева поређење идентификатора модула са тренутним контекстом превожња. Непоклапање идентификатора преусмерава претрагу на мапу увезених зависности. Принцип разрешавања симбола помоћу композитног кључа приказан је на слици 12.



Слика 12: Разрешавање симбола помоћу композитног кључа.

Независно превожње датотека захтева раздвајање фазе превожња од фазе повезивања модула у меморији. Нумерички идентификатори унутар учитаног модула захтевају усклађивање са адресним простором тренутног процеса. Систем покреће фазу релокације након учитавања зависности. Алгоритам за релокацију рекурзивно обилази стабло учитане датотеке. Компонента додељује нове вредности идентификаторима.

Примењени механизам опонаша концепт динамичког повезивања (енг. *dynamic linking*). Повезивање у меморији одржава конзистентност модела при раду са међузависним датотекама.

4.5.2 Перзистентност и реконструкција стања

Дизајн предвиђа чување анализираног модела на диску ради избегавања поновне анализе зависних датотека. Као формат серијализације изабран је бинарни запис без унапред одређене шеме. Унапред дефинисана структура је самодоволна приликом уписа на диск. Употребом бинарног формата сачувана је могућност проширења модела уз задржавање перформанси учитавања и уписивања података [13].

За потребе рада језичког сервера дизајниран је механизам трансформације модула назад у табелу симбола. Процес захтева итерацију кроз ставке унутар модула и мапирање њихових својстава. Реконструкција омогућава језичком серверу да креира контекст изолованих датотека. Сервер након реконструкције извршава валидацију међумодуларних веза.

4.6 Дизајн генератора података

Генератор података трансформише HIR у текстуалне податке. Синтезу покрећу извршне директиве за генерисање. Архитектура генератора обухвата:

- модул за евалуацију ограничења,
- модул за стохастичко узорковање и
- модул за очување референцијалног интегритета.

Подаци са листинга 12 су резултат генерације кода приказаног на листингу 13.

```
[
  {"sifra": "0D-0730"},
  {"id_radnika": "ZAP-58659", "odeljenje_id": "0D-0730"},
  {"id_radnika": "ZAP-13320", "odeljenje_id": "0D-0730"}
]
```

Листинг 12: Пример генерисаног JSON-а.

```
template Odeljenje {
  sifra = string_pattern "0D-#{[3]}";
}
template Zaposleni {
  id_radnika = string_pattern "ZAP-#{[4]}";
  odeljenje_id = ref Odeljenje.sifra;
}
@generate Odeljenje[1];
@generate Zaposleni[2];
```

Листинг 13: Спецификација шаблона и директива за генерисање података.

Модул за евалуацију ограничења анализира правила дефинисана над корисничким типовима. Вредности за променљиве изолују се из модела пре почетка синтезе. Анализатор мапира декларативна ограничења у математичке интервале и дефинисане скупове. Вредности унутар дефинисаних опсега генеришу се применом континуалне или дискретне униформне расподеле.

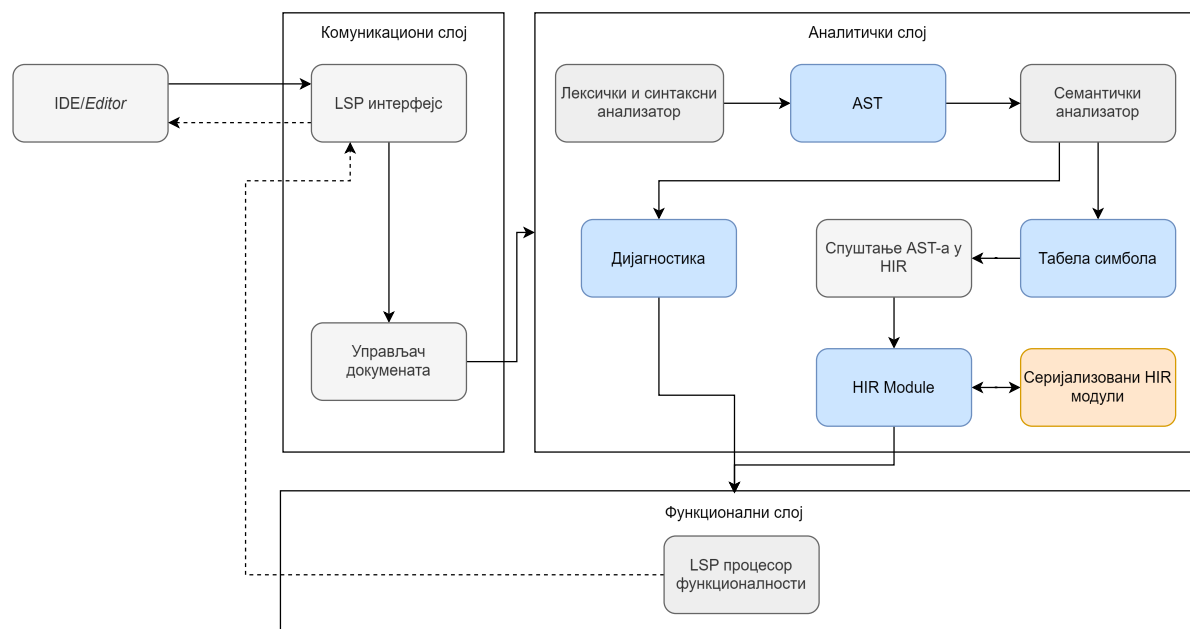
Модул за стохастичко узорковање управља синтезом случајних вредности и избором категорија. Израчунавање се заснива на PRNG. Иницијализација генератора почетном вредношћу обезбеђује детерминистички излаз и репродуктивност синтезе података [18]. Избор вредности из пробабилистичких енумерација изводи се алгоритмом за тежински избор. Свакој варијанти додељује се пропорционални део јединичног интервала. Псеудослучајна променљива одређује коначни избор према дефинисаним вероватноћама расподеле.

Модул за очување интегритета спречава појаву неконзистентних вредности генерисаних података. Зависне вредности захтевају изградњу оријентисаног графа зависности. Чворови графа представљају шаблоне генерације. Усмерене гране означавају смерове референцирања кроз израз *ref*. Тополошко сортирање графа

одређује редослед извршавања операција синтезе [34]. Процедура омогућава изградњу независних ентитета пре покретања генерације зависних објеката. Генератор преузима вредности из привременог бафера већ изгенерисаних кључева.

4.7 Дизајн језичког сервера

Језички сервер обухвата три логичка слоја: комуникациони слој (поглавље 4.7.1), аналитички слој (поглавље 4.7.2) и функционални слој (поглавље 4.7.3). Слика 13 приказује компоненте сваког слоја и ток обраде LSP захтева.



Слика 13: Архитектура LSP система и ток обраде LSP захтева.

4.7.1 Комуникациони слој

Комуникациони слој представља улазну тачку система. Задужен је за размену порука између уредника текста и језичког сервера. Слој прима LSP захтеве, управља стањем отворених докумената и прослеђује поруке осталим компонентама система [2].

Улазну тачку комуникационог слоја представља LSP интерфејс. Компонента прихвата захтеве које уредник текста шаље и преводи их у интерне операције система [2]. Након обраде, компонента формира одговор и враћа га уреднику текста [2].

Поред LSP интерфејса, комуникациони слој садржи управљач докумената (енг. *document manager*). Компонента прати измене над отвореним документима и одржава њихово стање за потребе даље обраде. Свака измена садржаја документа покреће поновну анализу и ажурирање дијагностичких порука.

4.7.2 Аналитички слој

Аналитички слој врши обраду DSL докумената и припрема податке потребне за LSP функционалности. Слој обухвата синтаксну анализу, семантичку анализу и изградњу интерних модела података. Резултат анализе представља HIR, која садржи семантички модел програма независан од синтаксног записа. Језичком серверу HIR пружа могућност симултаног приступа већем броју међусобно повезаних табела симбола.

Обрада започиње синтаксном анализом. Синтаксни анализатор трансформише текст документа у AST [6]. У случају уочавања синтаксних неправилности, синтаксни анализатор примењује механизме опоравка од грешака како би наставио са обрадом остатка документа. Овакав приступ омогућава систему да идентификује и прикаже више независних грешака истовремено, уместо прекида обраде при првој грешци [6].

Над генерисаним AST-ом извршава се семантичка анализа. Компонента проверава типове, валидност референци и конзистентност конструкција дефинисаних DSL правилима. Током анализе, формира се локална табела симбола, која садржи дефиниције и везе између елемената програма [6]. Уочене неправилности се бележе кроз модел дијагностике. Модел садржи опис грешке, ниво озбиљности и локацију у документу [2]. Након успешно завршене семантичке анализе, AST се трансформише у HIR. У HIR-у су разрешене референце, типови и односи између симбола из тренутног документа и из претходно серијализованих увозних модула.

4.7.3 Функционални слој

Функционални слој имплементира кориснички видљиве операције трансформацијом интерних модела у одговоре дефинисане LSP спецификацијом. Слој обухвата компоненте које омогућавају:

- аутоматско допуњавање кода на основу тренутног контекста и симбола доступних у локалној табели симбола и унутар серијализованих HIR модела увезених модула,
- дијагностику грешака, коришћењем резултата семантичке анализе за формирање порука о неправилностима,
- навигацију кроз код која кориснику обезбеђује прелазак на дефиниције симбола кроз глобални опсег дефинисан учитаним табелама симбола и
- приказ додатних информација (енг. *hover*), пружањем контекстуалних описа интеракције са кодом.

4.8 Дизајн командног интерфејса

Командни интерфејс представља улазну тачку приликом покретања система из терминала. Дизајн примењује образац диспечера команди [11]. Кориснички захтеви се прослеђују сервисима за анализу или превођење. Архитектура декомпонује систем на независне функционалне целине. Систем подржава интерактивни режим за појединачне захтеве и сервисни режим за комуникацију са развојним окружењима. Архитектура садржи скуп команди приказаних у табели 3.

Табела 3: Преглед доступних команди корисничког интерфејса

Команда	Опис	Параметри конфигурације
<i>generate</i>	Покретање синтезе тест података	Улазни модел, одредиште, формат записа, семе генератора
<i>compile</i>	Превођење и валидација модула	Улазни модел, излазна локација HIR структуре
<i>check</i>	Семантичка и синтаксна валидација кода	Скуп датотека за анализу
<i>init</i>	Иницијализација новог радног окружења	Локација пројекта, почетни шаблони
<i>info</i>	Приказ метаподатака о систему	Ниво детаљности приказа
<i>lsp</i>	Покретање језичког сервера	Тип комуникационог канала, подешавања дијагностике

4.9 Изазови и проблеми

Током пројектовања архитектуре система идентификовано је неколико изазова који су захтевали специфична дизајнерска решења. Први проблем односио се на стабилност синтаксне анализе у реалном времену. Алгоритми за синтаксну анализу прекидају извршавање приликом наилазак на прву неправилност. Прекидање рада није прихватљиво за рад унутар уредника текста где корисник константно пише непотпун код.

Проблем је решен увођењем токена за синхронизацију и механизма за опоравак од грешака (енг. *error recovery*) [6]. Систем у случају грешке изолује неисправан израз, конструише делимично стабло и наставља са рашчлањивањем остатка документа. Механизам опоравка од грешака омогућава приказивање више независних грешака истовремено.

Други проблем произашао је из асинхроне природе комуникације између уредника текста и сервера. Учестали захтеви за измену документа пристизали су брже него што је систем могао да заврши семантичку анализу претходног стања. Чести захтеви узрокују ризик од појаве услова трке (енг. *race condition*). Услов трке доводи до неконзистентности података унутар меморије преводиоца. За решавање проблема дизајниран је механизам за отказивање (енг. *cancellation token*) [2]. Механизам прекида активну нит позадинске анализе уколико из уредника пристигне нови унос. Применом механизма за отказивање ресурси сервера се преусмеравају на обраду најсвежијег стања кода.

Ово поглавље приказује имплементацију развијеног DSL-а и пратећег језичког сервера. Све компоненте система реализоване су у програмском језику *Rust*. Реализација прати фазе дефинисане у поглављу о дизајну. Лексички анализатор преводи изворни текст у ток токена (поглавље 5.1). Синтаксни анализатор трансформише ток токена у AST (поглавља 5.2 и 5.3). Компонента за семантичку анализу врши валидацију над генерисаним стаблом (поглавље 5.4). Систем преводи AST у HIR (поглавље 5.5).

Сваки модул развијен је као независна целина. Модули користе централизоване механизме за управљање грешкама и дијагностиком. Архитектура подржава размену података са језичким сервером. Исправност сваке компоненте потврђена је кроз јединичне и интеграционе тестове.

5.1 Лексичка анализа

Лексичка анализа представља прву фазу превођења. Циљ је трансформисати изворни текст у низ токена. Имплементација користи итератор *Peekable<CharIndices>*. Итератор омогућава обраду текста уз могућност прегледа наредних карактера без померања позиције показивача [35]. Увид у наредне карактере омогућава препознавање вишекарактерних оператора попут `&&`, `||`, `..` и `..=`. Препознавање се извршава без алокације додатног стања у меморији.

За препознавање кључних речи користи се регистар `KEYWORD_REGISTRY`. Регистар је имплементиран помоћу статичке мапе `once_cell::sync::Lazy`. Мапа обезбеђује константно време претраге приликом идентификације резервисаних речи [35]. Сваки токен чува позицију из изворног кода унутар структуре *Span*. Структура *Span* садржи почетну и крајњу линију и колону. Подаци о позицији омогућавају језичком серверу мапирање дијагностичких порука на оригинални текст.

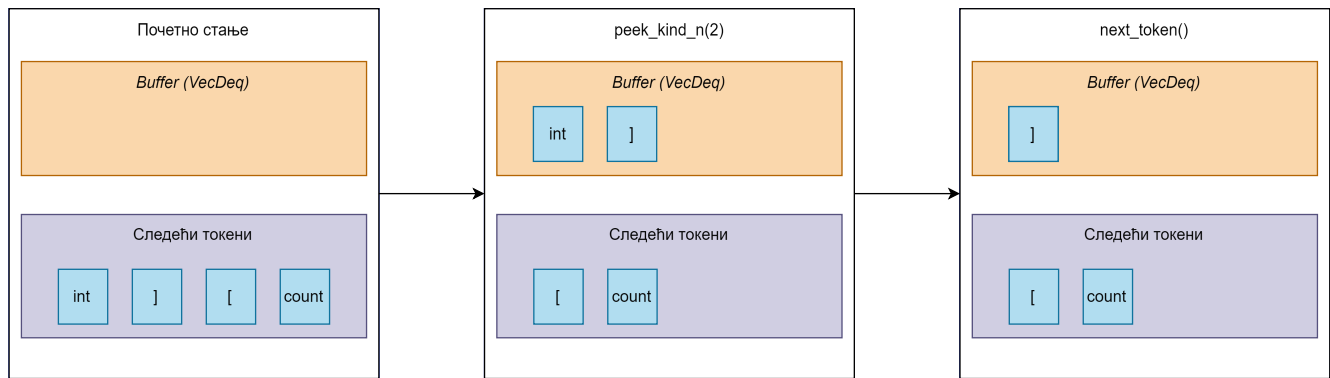
Структура *LexerError* служи за мапирање лексичких неправилности у дијагностичке поруке. При наиласку на невалидан карактер или нетерминирани литерал, лексички анализатор не прекида рад, већ генерише грешку и наставља обраду преосталог дела датотеке.

5.2 Синтаксна анализа

Синтаксни анализатор трансформише низ токена у AST. Имплементација је подељена на засебне датотеке попут *expression.rs*, *statement.rs* и *token_stream.rs*. Централна структура *Parser* управља целокупним процесом синтаксне анализе.

5.2.1 Управљање током токена

Компонента *TokenStream* омогућава предиктивну анализу и вирење унапред (енг. *lookahead*). Синтаксни анализатор прегледа наредне токене без њиховог уклањања из примарног тока. Преглед токена унапред неопходан је за разликовање литерала и типова података. Промена стања интерног бафера током вирења унапред приказана је на слици 14. Приликом позива операције за вирење унапред, прочитани токени се привремено смештају у интерни бафер, док их наредни позиви за читање преузимају из бафера.



Слика 14: Управљање интерним бафером током вирења унапред.

TokenStream користи итератор *Peekable* и интерни бафер *VecDeque* за привремено складиштење токена. Архитектура омогућава управљање стањем тока и прескакање токена током синхронизације. Поље *last_span* служи за праћење последње познате позиције у изворном коду. Структура компоненте је приказана на листингу 14.

```
pub struct TokenStream<I>
where
    I: Iterator<Item = Result<Token, LexerError>>,
{
    lexer: Peekable<I>,
    buffer: VecDeque<Token>,
    last_span: Span,
}
```

Листинг 14: Структура *TokenStream* компоненте.

Опоравак од грешака реализован је кроз метод *synchronize*. Синтаксни анализатор наставља рад након појаве грешке *ParserError*. Компонента прескаче токене унутар *TokenStream*-а до следећег поузданог граничника. Као граничници служе тачка-зарез или почетак новог исказа. Синхронизација омогућава пријаву више независних синтаксних грешака током једног покретања.

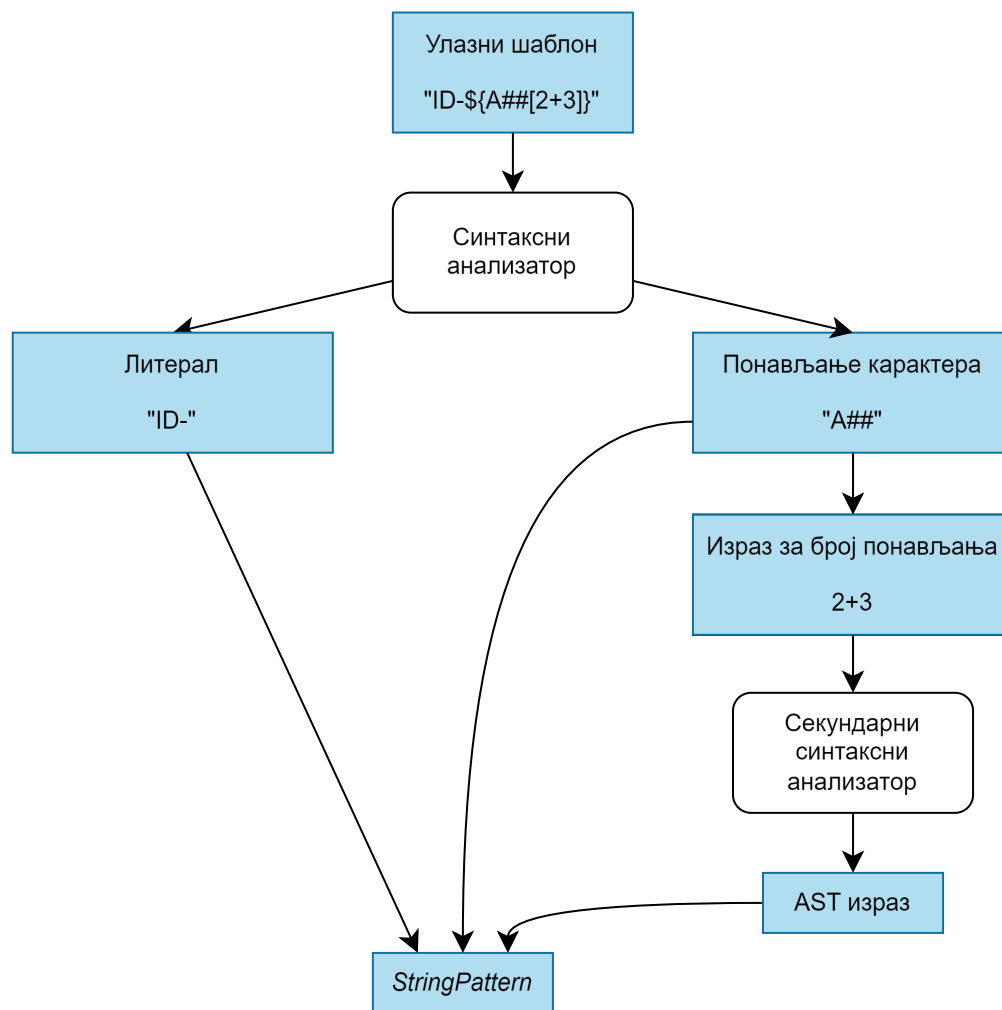
5.2.2 Рашчлањивање језичких конструкција

Обрада израза ослања се на Пратов синтаксни анализатор. Функција *parse_expression* прихвата аргумент *precedence* за одређивање приоритета оператора. Логика синтаксне анализе подељена је на две фазе. Прва фаза користи позив функције *parse_primary_expression* за обраду атомских вредности (литерали, идентификатори, заграде), док друга обухвата итеративну проверу наредног токена и поређење са тренутним нивоом приоритета. Синтаксни анализатор позива функцију *parse_infix_expression* за операције над већ обрађеним изразима.

Функција *parse_statement* усмерава ток извршавања на основу врсте токена. Директиве се обрађују помоћу методе *parse_directive*. Методе *parse_template* и *parse_struct* обрађују шаблоне и структуре. Методе *parse_enum* и *parse_type_declaration* обрађују еnumerације и декларације типова. Искази представљају заокружене семантичке целине попут дефиниција структура или шаблона. Изрази се обрађују унутар блокова *ExpressionStatement* са обавезном тачком-зарезом на крају.

Синтаксни анализатор обрађује типове података кроз систем наменских функција. Функција *parse_type* препознаје примитивне типове, листе и кориснички дефинисане типове. Систем подржава додавање семантичких ограничења на типове података. Ограничења се синтаксички дефинишу унутар угластих заграда. Регистар *CONSTRAINTS* садржи листу свих дозвољених врста ограничења. Подржана ограничења укључују опсег, минималну вредност, максималну вредност и дужину. Регистар користи статичку мапу за валидацију идентификатора ограничења током парсирања. Непозната ограничења изазивају прекид парсирања тренутног израза и генерисање грешке.

Језик подржава валидацију стрингова помоћу специјализованих текстуалних шаблона. Функција `parse_string_pattern` анализира садржај текстуалног литерала карактер по карактер. Синтаксни анализатор користи итератор `Peekable<CharIndices>` за навигацију кроз садржај шаблона. Механизам из литерала издваја константне делове и специјалне услове. Декомпозиција текстуалног шаблона током синтаксне анализе приказана је на слици 15. Услови дефинишу правила попут понављања малих слова, великих слова или бројева. Број понављања карактера може бити дефинисан динамички. Динамички број понављања захтева инстанцирање секундарног лексичког и синтаксног анализатора. Секундарни синтаксни анализатор обрађује угњеждане изразе унутар угластих заграда шаблона.



Слика 15: Декомпозиција текстуалног шаблона током синтаксне анализе.

5.3 Апстрактно синтаксно стабло

Систем имплементира AST као модуларну архитектуру унутар датотеке `ast/mod.rs`. Систем је подељен на логичке целине за изразе, исказе, варијанте, поља и ограничења. Централни чвор за репрезентацију израза у коду је структура `Expression`. Структура `Expression` и припадајућа еnumerација `ExpressionKind` приказане су на листингу 15.

```
pub struct Expression {
    pub kind: ExpressionKind,
    pub span: Span,
}
pub enum ExpressionKind {
    IntLiteral(usize),
    StringLiteral(String),
    Identifier(String),
    Type(DataType),
    Reference {
        template: String,
        field: String,
    },
    // ... остале варијанте су изостављене ради прегледности
}
```

Листинг 15: Структура и енумерација за репрезентацију језичких израза

Употреба паметних показивача (енг. smart pointer) *Box* омогућава рекурзивне чворове за префиксне и инфиксне операторе [35]. Структура *Element* омогућава рад са листама. Структура програма и синтаксне наредбе представљене су кроз енумерацију *Statement*. Свака варијанта унутар енумерације *Statement* чува метаподатке о позицији и структури изворног кода. Искази који садрже самосталан израз издвајају се унутар наменске структуре *ExpressionStatement*.

За потребе претраге, валидације и трансформације генерисаног стабла развијен је интерфејс *Visitor*. Интерфејс дефинише методе за посету сваког појединачног типа чвора у AST-у. Функције *walk_program*, *walk_statement* и *walk_expression* омогућавају рекурзивно кретање кроз дубину стабла. Помоћне функције усмеравају посетиоца на угњежене подчворове израза, варијанти и поља. Интерфејс олакшава додавање нових пролаза за анализу без измене структуре стабла.

5.4 Семантичка анализа

Семантичка анализа врши контекстуалну валидацију AST-а након синтаксне анализе. Процес осигурава испуњеност правила која превазилазе могућности контекстно-слободних граматика. Архитектура система заснива се на вишефазном приступу. Имплементација користи дизајнерски образац посетилац за обилазак језичких конструкција. Структура *SemanticAnalyser* <*a*> оркестрира процес анализе кроз методе *analyse* и *analyse_with_imports*. Систем као резултат генерише структуру *AnalysisResult*. Излазна структура садржи табелу симбола, мапу референци, мапу типова и колекцију дијагностичких порука.

5.4.1 Фазе валидације и управљање контекстом

Процес семантичке анализе обухвата пет секвенцијалних фаза.

- Прва фаза подразумева изградњу табеле симбола кроз структуру *SymbolTableBuilder*. Компонента региструје глобалне и локалне симболе уз успостављање хијерархије опсега важења. Детекција дуплираних декларација прекида процес анализе пре покретања наредних фаза.
- Друга фаза извршава детекцију циклуса у наслеђивању шаблона. Систем проверава постојање кружних зависности пре почетка валидације поља и референци.
- Трећа фаза обухвата проверу референци помоћу компоненте *ReferenceChecker*. Анализатор потврђује постојање сваког идентификатора у тренутном контексту или унутар увезених модула.
- Четврта фаза мапира међусобне везе објеката кроз структуру *ReferenceTracker*. Прикупљени метаподаци служе за очување референцијалног интегритета током генерисања података.
- Пета фаза извршава проверу типова и ограничења помоћу компоненте *TypeChecker*. Систем закључује типове израза, детектује некомпатибилност оператора и валидира семантичку исправност ограничења.

Структура *SymbolTable* управља симболима кроз организацију стабла повезаних опсега. Сваки опсег садржи идентификатор *ScopeId*, мапу локалних симбола и показивач ка родитељском опсегу. Лексичко претраживање симбола од локалног ка глобалном нивоу извршава се позивом методе *lookup*.

Систем дефинише врсте опсега кроз енумерацију *ScopeKind*. *Global* је врховни опсег за декларације шаблона, структура, енумерација и типова. *Template* дефинише опсег унутар једног шаблона. *Enum* ограничава контекст унутар дефиниције енумерације. *Generate* означава опсег унутар блока за генерацију података. *Directive* и *Block* служе као помоћни локални опсези. Симболи су представљени структуром *Symbol*. Полиморфна структура за чување метаподатака о језичким конструктима приказана је на листингу 16.

```
pub enum SymbolKind {
    Template {
        fields: Vec<String>,
        parent: Option<String>,
    },
    Struct {
        name: String,
        fields: Vec<Field>,
    },
    Enum {
        variants: Vec<VariantInfo>,
    },
    TypeAlias {
        name: String,
        data_type: Expression,
    },
    Field {
        template_name: String,
        is_override: bool,
        expression: Expression,
    },
}
```

Листинг 16: Енумерација *SymbolKind* за репрезентацију семантичких метаподатака

Структура *SymbolTable* имплементира алгоритам за детекцију циклуса унутар методе *check_inheritance_cycle*. Алгоритам спречава бесконачну рекурзију приликом кружног наслеђивања шаблона. Претрага пролази кроз ланац наслеђивања праћењем референци ка родитељима. Алгоритам користи хеш-скуп за праћење посећених чворова. Поновно појављивање истог шаблона унутар тренутне путање прекида извршавање претраге. Метода враћа листу стрингова са тачним редоследом цикличне зависности. Анализатор трансформише резултат у дијагностичку грешку *SemanticError::InheritanceCycle*.

Компонента *TypeChecker* извршава валидацију типова израза унутар AST-а. Компонента извршава валидацију тежинских фактора унутар дефиниција енумерација. Метод *visit_variant* захтева целобројни тип израза за дефинисање тежине. Одступање од целобројног типа резултује грешком о неслагању типова. Анализатор контролише команде за генерацију података кроз метод *visit_generate*. Израз за дефинисање броја генерисаних инстанци мора припадати целобројном типу. Систем анализира изразе ограничења (нпр. *Range*, *Min* и *Max*) и проверава њихову компатибилност са тренутним типом података.

Семантички анализатор спроводи стратегију акумулације грешака уместо прекидања извршавања при првој неправилности. Систем чува све пронађене нерегуларности унутар колекције грешака структуре *SemanticAnalyser*. Семантичке грешке су типизирани кроз енумерацију *SemanticError*. Дефиниција типизираних семантичких грешака и метод за њихову трансформацију у дијагностички формат приказани су на листингу 17.

```

pub enum SemanticError {
    UnknownIdentifier {
        name: String,
        span: Span,
    },
    TypeMismatch {
        expected: String,
        found: String,
        span: Span,
    },
    // ... остале варијанте грешака
}

impl SemanticError {
    pub fn to_diagnostic(&self) -> Diagnostic {
        match self {
            SemanticError::TypeMismatch { expected, found, span } => {
                Diagnostic::error(
                    *span,
                    format!("Type mismatch: expected '{}', found '{}'", expected, found),
                )
                .with_code(DiagnosticCode::TypeMismatch)
                .with_hint("Ensure the expression matches the expected type")
            }
            // ... мапирање осталих грешака
        }
    }
}

```

Листинг 17: Дефиниција семантичких грешака и мапирање у објекте дијагностике

Свака варијанта садржи структуру *Span* за мапирање координата у изворном коду. Метода *to_diagnostic* трансформише објекат семантичке грешке у LSP дијагностички формат. Систем генерише контекстуалне савете за исправку на основу специфичног типа грешке.

5.5 Високонивојска међурепрезентација

HIR је упрошћена форма програма након завршене семантичке анализе. Трансформација из AST-а у HIR уклања синтаксне конструкције и нормализује структуру података. HIR служи као основа за даљу оптимизацију, проверу компатибилности и серијализацију модула. Инфраструктура за управљање меморијом и дијагностиком приказана је на листингу 18.

```

pub struct StringPool {
    strings: Vec<String>,
    map: HashMap<String, StringId>,
}

pub struct SpanMap<K, V> {
    entries: Vec<(K, V)>,
    cache: OnceLock<HashMap<K, V>>,
}

```

Листинг 18: Инфраструктура за меморијску оптимизацију и мапирање изворног кода

Интернирање стрингова преко структуре *StringPool* обезбеђује репрезентацију идентификатора. *SpanMap* омогућава повезивање метаподатака са позицијама у изворном коду. Структура користи *OnceLock* за лењо кеширање претрага ради убрзања приступа координатама. Поступак трансформације изводи компонента *AstLowering*. Имплементација структуре *AstLowering* приказана је на листингу 19.

```
pub struct AstLowering {
    context: LoweringContext,
    string_pool: StringPool,
    source_map_builder: SourceMapBuilder
}
```

Листинг 19: Структура *AstLowering* задужена за трансформацију AST-a у HIR

Компонента *AstLowering* користи *string_pool* за дедупликацију текста и *source_map_builder* за праћење порекла чворова. Метода *lower_expr* рекурзивно обилазе чворове AST-a. Генерисани модел омогућава бинарну серијализацију модула.

5.6 Систем модула

Систем модула омогућава декомпозицију програма у независно преводиве целине. Сваки модул представља јединицу преводјења која се, након анализе, серијализује у бинарни формат *.tmod*. Архитектура подржава увоз симбола кроз систем референци и мапирање идентификатора.

5.6.1 Разрешавање зависности и глобално адресирање

Униформно глобално адресирање апстрахује физичку локацију ставки. Идентификација симбола у меморији ослања се на композитне нумеричке кључеве (листинг 20).

```
pub struct LocalItemId(pub u32);
pub struct ModuleId(pub u32);
pub struct GlobalItemId {
    pub module: ModuleId,
    pub item: LocalItemId,
}
```

Листинг 20: Структуре података за глобално адресирање ставки

Структура *LocalItemId* одређује ставку унутар граница једне датотеке. Структура *GlobalItemId* спаја локални идентификатор са идентификатором модула (*ModuleId*). Глобално адресирање омогућава униформан рад са локалним и спољашњим симболима.

Компонента *ModuleResolver* управља процесом проналажења и учитавања зависности у меморију. Претрага обухвата директоријуме стандардне библиотеке и локалне путање пројекта. Имплементација методе за разрешавање зависности приказана је на листингу 21.

```
pub struct ModuleResolver {
    search_paths: Vec<PathBuf>,
    cache: HashMap<String, Module>,
    next_module_id: u32,
}
impl ModuleResolver {
    fn resolve_one(&mut self, name: &str, path: &Path) -> Result<(), ResolveError> {
        if self.cache.contains_key(name) { return Ok(()); }
        let path = self.find_tmod(name, path)?;
        let mut module = Module::load(&path)?;
        let allocated_id = ModuleId(self.next_module_id);
        self.next_module_id += 1;
        self.cache.insert(name.to_string(), module);
        Ok(())
    }
}
```

Листинг 21: Логика разрешавања, валидације кеша и релокације модула.

ModuleResolver користи хеш-мапу за мемоизацију. Мемоизација спречава поновно учитавање истог модула. У случају неуспешне резолуције или грешке при читању, систем генерише *ResolveError* са подацима о узроку.

5.6.2 Серијализација

Библиотека *rpm_serde* служи за бинарно форматирање података у *MessagePack* формат [36]. Бинарни запис убрзава учитавање и омогућава инкрементално превођење. Фаза превођења се прескаче уколико изворни код није мењан, што систем проверава поређењем хеш вредности тренутног садржаја са вредношћу сачуваном у метаподацима.

Процес серијализације обухвата целокупну структуру *Module*, укључујући припадајући *StringPool* и све дефинисане ставке. Глобално адресирање и интернирање стрингова, спроведено у претходним фазама, осигурава очување референцијалног интегритета након десеријализације. Учитани подаци постају спремни за генерисање излазних формата или анализу унутар језичког сервера.

5.7 Генерисање података

Генерисање података ослања се на концепт лење евалуације (енг. *lazy evaluation*). Структура *RecordGenerator* имплементира стандардни интерфејс итератора [35]. Итератор генерише записе секвенцијално на експлицитан захтев. Систем обрађује листу директива и конструише појединачне записе преко функције *generate_record*. Алгоритам користи интерни евалуатор за динамичко израчунавање вредности поља током итерације. Имплементација елиминише потребу за учитавањем комплетног скупа података у радну меморију.

Особина (енг. *trait*) *FileGenerator* (листинг 22) дефинише апстракцију за серијализацију генерисаних структура. Интерфејс захтева дефинисање екстензије датотеке и имплементацију методе за генерисање појединачног записа. Опционе методе омогућавају додавање заглавља, подножја и сепаратора између записа. Систем изворно подржава упис у CSV, JSON, XML и SQL формате кроз засебне имплементације дефинисане особине. Сваки формат подржава динамичку инстанцијацију путем конфигурационе мапе објеката. Конфигурација омогућава прилагођавање атрибута попут сепаратора, увлачења текста или имена SQL табеле. Функција *write_records* прихвата итератор и уписује форматиране линије у излазну датотеку.

```
pub type Record = IndexMap<String, Object>;
pub trait FileGenerator: fmt::Debug {
    fn generate(&self, record: &Record) -> Result<String, GeneratorError>;
    fn extension(&self) -> &str;
    fn generate_header(&self, fields: &[String]) -> Option<String> { None }
    fn generate_footer(&self) -> Option<String> { None }
    fn needs_separator(&self) -> bool { false }
    fn separator(&self) -> &str { "" }
}
```

Листинг 22: Дефиниција особине *FileGenerator* за генерисање излазних формата.

5.8 Језички сервер

Комуникација са развојним окружењима одвија се кроз LSP. Сервер омогућава анализу изворног кода, навигацију кроз дефиниције и преглед упозорења у реалном времену. Имплементација користи библиотеку *tower-lsp* [37].

Структура *Workspace* управља документима. Приступ садржају датотека обезбеђује механизам за синхронизацију *RwLock*. Документ обједињује текст, верзију и структуру *Analysis*. Анализа садржи AST, табелу симбола, референце и листу дијагностичких порука.

Компонента *AnalysisEngine* координише превођење:

- лексичку и синтаксну анализу,
- резолуцију увоза модула кроз *ModuleResolver* и

- семантичку анализу путем *SemanticAnalyser*-а.

Метода *update* (листинг 23) повезује фазе компилације са LSP догађајима. Анализа се покреће при свакој измени текста. Резултати се мапирају у структуру документа, док се дијагностичке поруке превode у LSP формат. Раздвајање логике анализе од приказивања омогућава приказ грешака у едитору без прекида рада сервера.

```
pub async fn update(
    &self,
    uri: Url,
    text: String,
    version: i32
) -> Vec<lsp_types::Diagnostic> {
    let file_path = uri.to_file_path().ok();
    let result = AnalysisEngine::analyse(&text, file_path.as_deref());

    let document = match (result.ast, result.symbol_table) {
        (Some(ast), Some(symbols)) => {
            Document::new(uri.clone(), text, version).with_analysis(Analysis {
                // ... креирање анализе
            })
        }
        _ => Document::new(uri.clone(), text, version),
    };

    self.insert(uri.clone(), document).await;

    result.diagnostics.into_iter()
        .map(Diagnostic::to_lsp_diagnostics)
        .collect()
}
```

Листинг 23: Ажурирање документа и синхронизација анализе

5.9 Командни интерфејс

Реализација CLI-а користи библиотеку *clap* [38]. Библиотека омогућава декларативно дефинисање аргумената кроз *Rust* структуре и енумерације. Мапирање корисничког улаза у типске вредности елиминише потребу за ручним парсирањем стрингова. Систем детектује неисправне уносе током фазе превођења кода, што повећава поузданост програма.

Архитектура почива на енумерацији *Command*. Енумерација дефинише све доступне операције система. Свака варијанта поседује структуру аргумената. Дизајн омогућава јасну декомпозицију логике (листинг 24).

```
#[derive(Parser)]
pub struct Cli {
    #[command(subcommand)]
    pub command: Command,
}
#[derive(Subcommand)]
pub enum Command {
    Generate { ... },
    Compile { ... },
    // ... остале CLI команде
}
```

Листинг 24: Структура CLI команди

Функција *run* координише извршавање. Систем акумулира *Diagnostic* поруке приликом појаве грешака. Поруке се форматирају за корисника. Програм враћа повратни код за интеграцију са системским алатима.

У овом поглављу представљени су резултати испитивања перформанси развијених програмских решења. Анализа обухвата:

- временску ефикасност фаза превођења (поглавље 6.2),
- утицај система модула на брзину превођења (поглавље 6.3),
- временску и меморијску ефикасност генерисања података (поглавље 6.4) и
- време одзива језичког сервера (поглавље 6.5).

6.1 Експериментално окружење

Мерења су спроведена у контролисаном хардверском и софтверском окружењу ради добијања репродуктивних резултата. Сви тестови перформанси извршени су на рачунару конфигурације:

- Процесор: Intel Core i7-11800H радног такта 2,30 GHz
- Радна меморија: 16 GB DDR4
- Оперативни систем: Windows 10 Professional

Тестирање временске ефикасности процеса превођења и језичког сервера реализовано је применом библиотеке *Criterion* унутар програмског језика *Rust*. Мерења извршена су над програмом преведеним у *release* режиму. *Criterion* користи методу статистичког узорковања са претходним фазама загревања како би се ублажио утицај позадинских процеса оперативног система и стања кеш меморије. Свако појединачно мерење извршено је кроз стотину узастопних понављања.

6.2 Евалуација процеса превођења

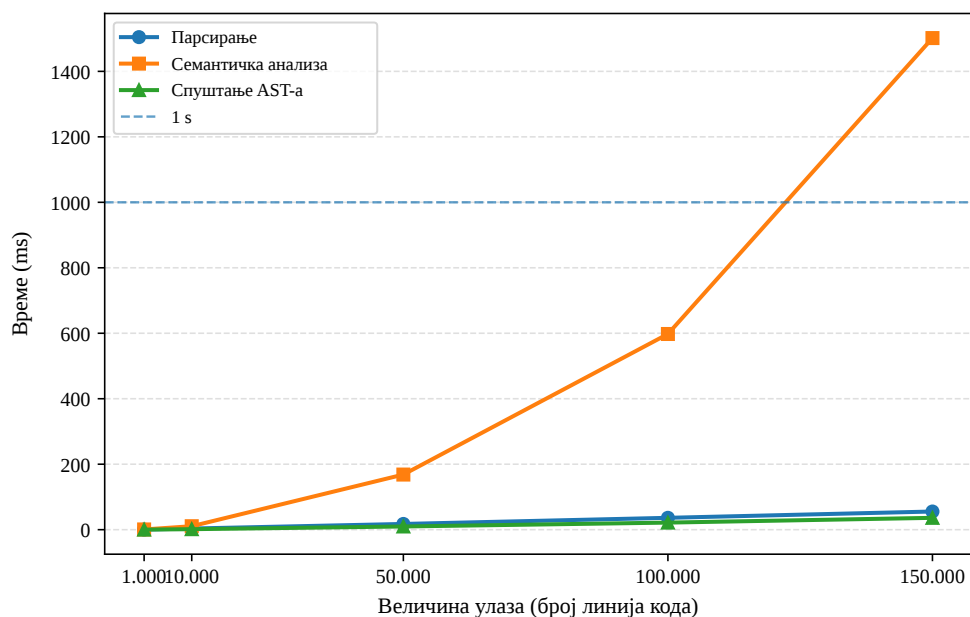
Евалуација процеса превођења спроведена је ради утврђивања времена потребног за извршавање појединачних фаза превођења при раду са улазним датотекама различитих величина. Тестирање је обухватило синтаксно исправне датотеке величине од 1.000 до 150.000 линија кода. Испитиване су:

- лексичка и синтаксна анализа,
- семантичка анализа,
- процес спуштања.

Критеријум прихватљивости захтева укупно време превођења испод 1.000 ms за датотеке величине до 100.000 линија кода. Испуњење критеријума омогућава несметан интерактивни рад унутар развојних окружења, где језички сервер пружа брзу повратну информацију без приметног кашњења [3]. Непожељан исход представља прекорачење границе од 1.000 ms за средње величине датотека. Прекорачењем границе се нарушава континуитет рада корисника услед кашњења при анализи кода. Средње време извршавања фаза превођења садржи табела 4. График на слици 16 приказује зависност времена од величине датотеке.

Табела 4: Средње време извршавања фаза процеса превођења.

Број линија кода	Формирање AST (ms)	Семантика (ms)	Спуштање (ms)	Укупно (ms)
1.000	0,40	0,69	0,18	1,27
10.000	3,50	10,58	1,50	15,58
50.000	17,51	168,35	9,75	195,61
100.000	36,23	597,76	21,59	655,58
150.000	55,46	1501,70	36,14	1593,30



Слика 16: Зависност времена извршавања фаза превођења од броја линија кода

Мерења указују на испуњење дефинисаног критеријума успеха за датотеке обима до 100.000 линија кода. Укупно време превођења датотеке од 50.000 линија износило је 195,61 ms. Време превођења датотеке од 100.000 линија износило је 655,58 ms. Измерена времена налазе се испод дефинисане границе од 1.000 ms. Лексичка и синтаксна анализа показале су линеарни раст времена извршавања у односу на величину улаза. Синтаксна анализа датотеке обима 150.000 линија извршавала се за 55,46 ms. Процес спуштања за датотеку обима 150.000 линија захтевао је 36,14 ms.

Укупно време превођења за датотеку од 150.000 линија износило је 1593,30 ms. Измерена вредност прелази границу од 1.000 ms дефинисану за интерактивни рад. Прекорачење времена извршавања последица је нелинеарне сложености унутар фазе семантичке анализе. Време семантичке провере расло је са 597,76 ms за 100.000 линија на 1501,70 ms за 150.000 линија. Добијени резултат дефинише границу употребљивости тренутне имплементације. Идентификовано ограничење указује на потребу за развојем алгорита за инкременталну семантичку анализу у оквиру будућег рада.

6.3 Евалуација система модула

Евалуација система модула квантификује трошак механизма за разрешавање зависности. Трошак обухвата читавање датотека и динамичку релокацију симбола. Повећани број улазно-излазних операција генерише оптерећење система. Критеријум успешности је линеарна сложеност $\mathcal{O}(N)$ при читавању N јединица превођења. Непожељан исход је експоненцијални раст времена превођења.

Експеримент је анализирао три топологије стабла зависности у величинама од 10, 25, 50, 75 и 100 јединица:

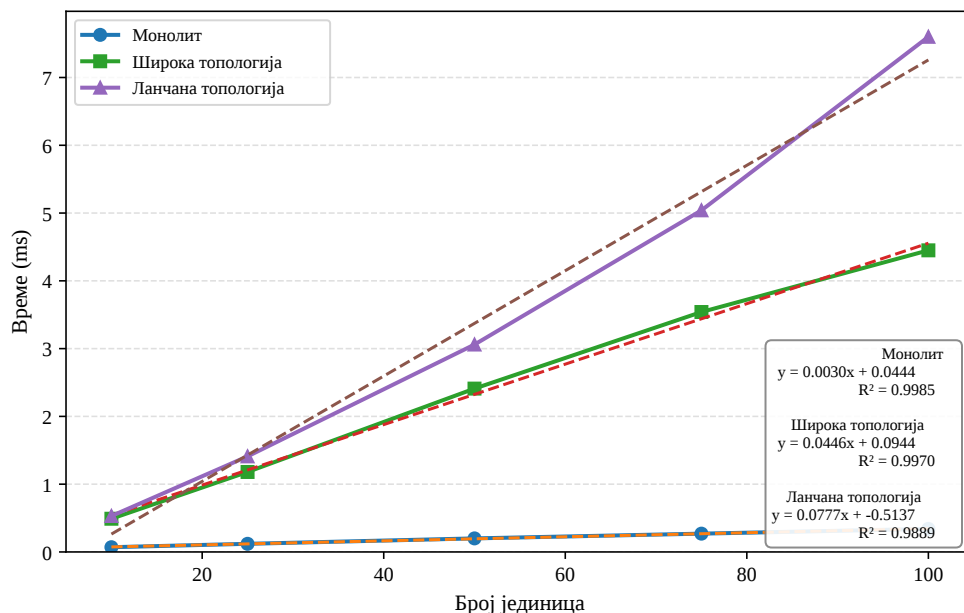
- монолитна база, једна датотека садржи све дефиниције. Мерење изолује брзину десеријализације без спољних датотека;
- широка топологија, главни модул увози N независних модула. Дубина стабла износи један. Мерење изолује трошак отварања датотека; и
- ланчана топологија, сваки модул увози тачно једну наредну јединицу. Дубина стабла износи N . Топологија оптерећује рекурзивни алгоритам за релокацију.

Резултати евалуације су приказани у табели 5.

Табела 5: Време учитавања и разрешавања зависности у односу на топологију модула.

Број јединица	Монолит (ms)	Широка топологија (ms)	Ланчана топологија (ms)
10	0,07	0,49	0,53
25	0,12	1,18	1,41
50	0,20	2,41	3,06
75	0,27	3,54	5,04
100	0,34	4,45	7,60

Подаци потврђују линеарно скалирање алгоритма. Монолитна база учитавала је 100 ставки за 0,34 ms. Резултат рефлектује извршавање једног системског позива. Широка топологија учитавала је 100 јединица за 4,45 ms. Разлика у времену представља трошак отварања појединачних датотека. Ланчана топологија захтевала је 7,60 ms за 100 јединица. График на слици 17 приказује линеарни тренд времена превођења у функцији броја модула. Линеје регресије за сваку топологију потврђују да систем одржава константан однос раста у односу на број улазних јединица. Одступања података од линеје регресије су занемарљива, што потврђује претпостављену линеарну сложеност $\mathcal{O}(N)$.



Слика 17: Линеарна зависност времена превођења од броја модула за различите топологије.

6.4 Евалуација генерисања података

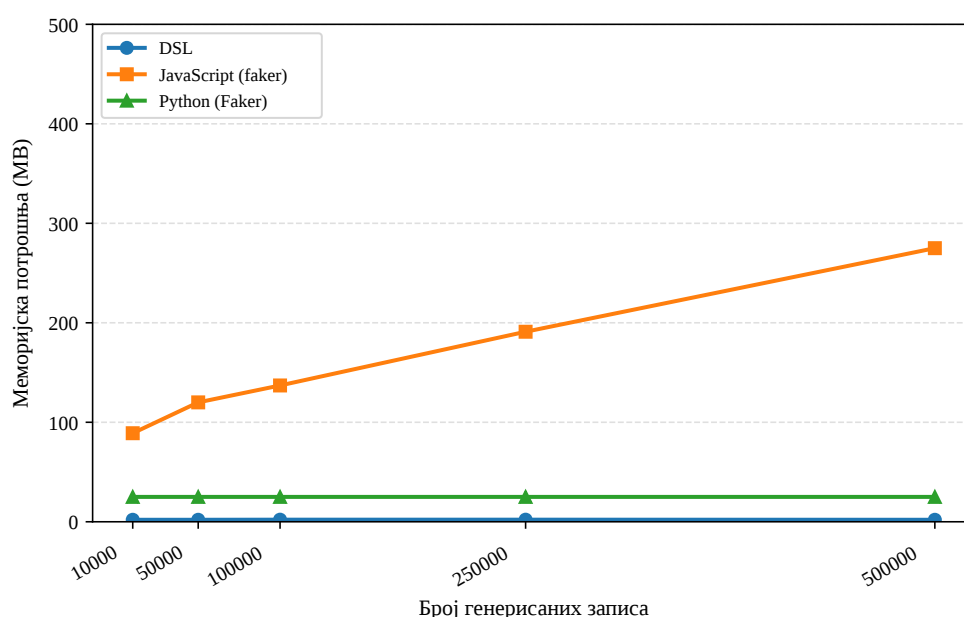
Евалуација генерисања података квантификује пропусну моћ и меморијску ефикасност система. Критеријум успешности имплементације захтева задржавање просторне сложености на нивоу $\mathcal{O}(1)$. Временска сложеност процеса генерисања мора имати линеарни раст $\mathcal{O}(N)$.

Експеримент пореди развијени систем са референтним библиотекама у језицима *Python (Faker)* (верзија 3.12.3) и *JavaScript (@faker-js/faker)* (NodeJS верзија 18.19.1). Тест модел представља равну структуру од десет поља различитих типова података. Одсуство релационих зависности између записа омогућава изолацију перформанси лење евалуације. Мерење обухвата генерисање скупа од 10.000, 50.000, 100.000 и 500.000 CSV записа. Меморијска потрошња се бележи на нивоу оперативног система праћењем максималне величине резидентног скупа (енг. *Peak RSS*). Резултати тестирања приказани су у табели 6.

Табела 6: Поређење времена генерисања и меморијске потрошње различитих имплементација.

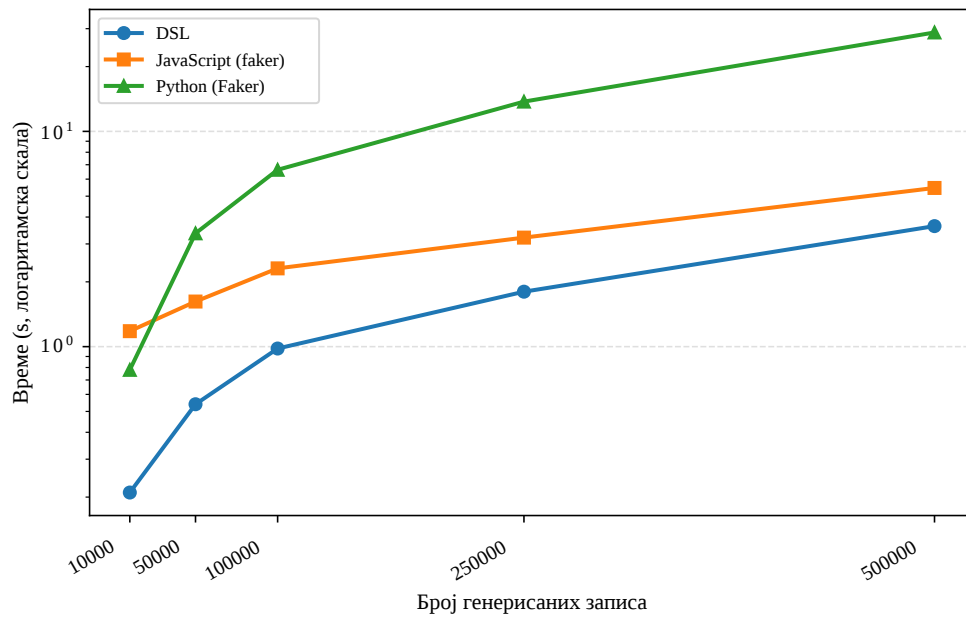
Број записа	Време (DSL)	Мем. (DSL)	Време (JS)	Мем. (JS)	Време (Python)	Мем. (Python)
10.000	0,21 s	2,0 MB	1,18 s	89 MB	0,78 s	25 MB
50.000	0,54 s	2,0 MB	1,62 s	120 MB	3,36 s	25 MB
100.000	0,98 s	2,1 MB	2,31 s	137 MB	6,63 s	25 MB
250.000	1,80 s	2,1 MB	3,21 s	191 MB	13,75 s	25 MB
500.000	3,63 s	2,0 MB	5,46 s	275 MB	28,81 s	25 MB

Експериментални резултати потврђују боље перформансе развијеног система у поређењу са референтним скриптним језицима. Развијени систем је задржао потрошњу меморије од 2 MB при генерисању 500.000 записа. Статична потрошња доказује просторну сложеност $\mathcal{O}(1)$ [34]. Имплементација у *Node.js* окружењу бележила је линеаран раст меморије. Системска потрошња је достигла 275 MB при генерисању 500.000 записа. График на слици 18 приказује меморијску зависност од броја записа.



Слика 18: Зависност меморијске потрошње од броја генерисаних записа.

Развијени систем остварује најмање време извршавања у свим сценаријима. Генерисање 500.000 записа трајало је 3,63 s. Резултат потврђује ефикасност директног уписа на диск без посредничких структура података. Време извршавања *Node.js* скрипте при генерисању 10.000 записа износило је 1,18 s. Перформансе *Python* скрипте линеарно су опадале при повећању обима података. Време извршавања скупа података од 500.000 записа у *Python* окружењу износило је 28,81 s. График на слици 19 приказује зависност времена од броја записа.



Слика 19: Зависност времена генерисања од броја записа.

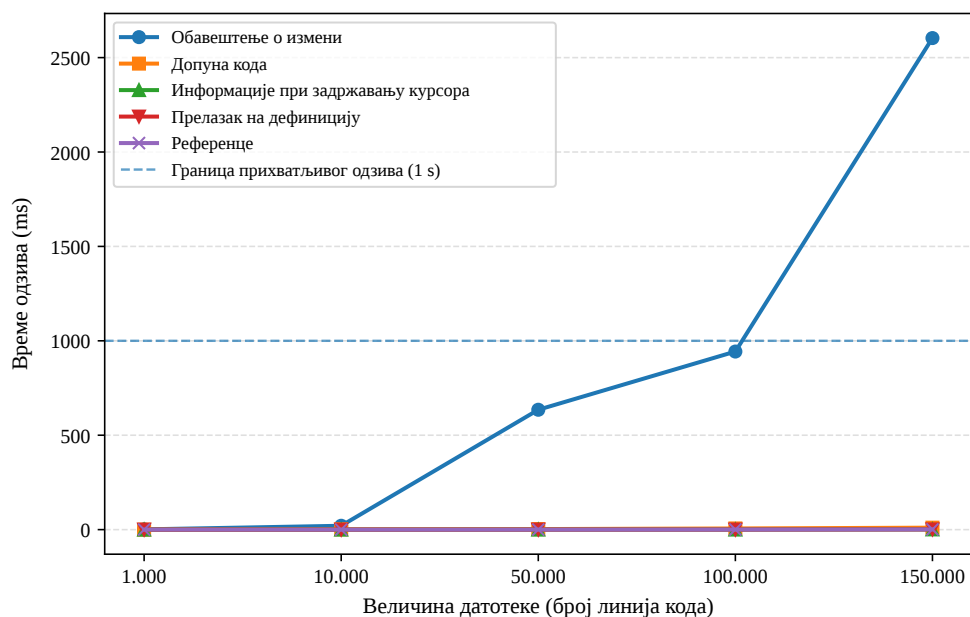
6.5 Евалуација језичког сервера

Циљ евалуације језичког сервера био је испитивање утицаја величине датотеке на време одзива језичког сервера. Пожељан исход је време одзива мање од 100 ms. Временски оквир мањи од 100 ms омогућава кориснику осећај тренутне реакције система [39]. Граница прихватљивости износи 1 s, након чега долази до прекида тока мисли корисника [39]. Непожељан исход је време одзива дуже од 1 s.

Експеримент је спроведен мерењем времена одзива LSP захтева унутар контролисаног тестног окружења. Тестирање је извршено над пет синтаксно исправних DSL датотека различитих величина. Величине датотека износиле су 1.000, 10.000, 50.000, 100.000 и 150.000 линија кода. За сваку датотеку је измерено време одзива пет различитих операција језичког сервера. Анализиране операције обухватале су:

- измену документа,
- приказ информација,
- прелазак на дефиницију,
- претрагу референци и
- аутоматско допуњавање кода.

Резултати мерења приказани су на слици 20. Прикупљени подаци указали су на постојање две категорије перформанси. Категорије су дефинисане на основу архитектонске реализације операција. Прву категорију су чиниле функционалности засноване на претрази табеле симбола. Операције су задржале константно време одзива од приближно 1 ms на целом опсегу тестирања. Друга категорија је обухватала операцију измене документа која захтева поновну обраду текста. Поновна обрада текста је показала линеаран раст времена одзива са повећањем величине датотеке. При раду са датотеком од 1.000 линија кода време одзива је износило 1,29 ms. На датотеци од 150.000 линија кода време одзива достигло је 2.603,50 ms.



Слика 20: Зависност времена одзива LSP операција од величине датотеке.

Синтеза спроведене евалуације потврдила је зависност укупних перформанси од примењеног начина синтаксне анализе. Операције над табелом симбола задржале су високу ефикасност кроз цео експеримент. Време одзива на датотеци од 150.000 линија кода премашило је прихватљив праг одзива од 1 s. Синтаксна анализа је узрок пада скалабилности архитектуре. Имплементација алгоритма за инкременталну синтаксну анализу је наредни корак у развоју алата.

6.6 Дискусија резултата

Резултати евалуације потврђују ефикасност развијеног система. Архитектура испуњава критеријуме прихватљивости за датотеке до 100.000 линија кода. Развијено решење превазилази перформансе референтних алата при генерисању података.

Директан упис на диск одржава константну потрошњу меморије. Систем заузима 2 MB простора независно од обима излазних података. Добијена вредност потврђује просторну сложеност $\mathcal{O}(1)$. Механизам модула линеарно скалира. Операције језичког сервера над табелом симбола остварују време одзива од 1 ms. Измерене вредности омогућавају несметан рад корисника у развојном окружењу.

Тестирање открива ограничења архитектуре. Обрада датотека већих од 100.000 линија деградира перформансе. Време извршавања семантичке провере расте нелинеарно. Операција измене документа захтева синтаксну анализу текста при сваком уносу карактера. Одзив језичког сервера прелази границу од 1 s за масивне датотеке.

Уочена ограничења дефинишу правац даљег развоја система. Оптимизација перформанси захтева инкрементални модел обраде AST-а. Инкрементална анализа представља надоградњу за рад са великим монолитним датотекама. Систем успешно интегрише концепте компилаторске теорије и генератора података.

У оквиру овог рада дизајниран је и имплементиран екстерни DSL за генерисање тест података и пратећи језички сервер. Анализа постојећих решења указала је на њихова ограничења. Алати за синтезу података најчешће захтевају писање императивног кода или коришћење затворених графичких платформи. Предложено решење успешно превазилази ограничења увођењем декларативне текстуалне синтаксе.

Технички доприноси рада обухватају:

- развој преводаца у програмском језику *Rust* са лексичком, синтаксном и семантичком анализом;
- успостављање модларне архитектуре засноване на HIR-у и бинарној серијализацији ради убрзања компилације;
- имплементацију стохастичког генератора са подршком за статистичке расподеле и очување референцијалног интегритета и
- развој језичког сервера за интеграцију са развојним окружењима путем LSP-а.

Дизајн доказује изводљивост креирања специјализованог алата. Развијени систем пружа статичку анализу у реалном времену, уз задржавање перформанси приликом локалног извршавања.

Архитектура система поставља основу за даља проширења. Модуларни дизајн предњег и задњег дела преводиоца омогућава идентификацију праваца за даљи развој.

8.1 Унапређење система превођења

Тренутна имплементација захтева ручно покретање преводиоца за сваки модул. Будућа верзија система требало би да уведе механизам за инкрементално превођење који би аутоматизовао процес превођења. Систем би пратио измене у изворним *.testa* датотекама помоћу механизма за праћење промена (енг. *file watchers*) и поредио временске ознаке (енг. *timestamps*) изворног кода са постојећим бинарним модулима.

Приликом детекције измене, преводац би ажурирао само оне модуле који су измењени или који зависе од измењеног кода. Интеграција механизма детекција измене уклонила би потребу за мануелном интервенцијом и осигурала конзистентност између изворног и преведеног кода током развоја.

8.2 Средњенивојска међурепрезентација

Тренутна архитектура преводи изворни код директно из AST структуре у HIR. Имплементација MIR-а омогућила би увођење додатних фаза оптимизације. MIR служи као канонички приказ програма у којем се сложени изрази декомпонују у низ једноставних, линеарних операција.

Увођење MIR-а отвара могућност за имплементацију анализа попут анализе тока података (енг. *data-flow analysis*) и елиминације „мртваг кода“ (енг. *dead code elimination*). Прелазак на вишестепену архитектуру IR-а (HIR за семантичку анализу, MIR за оптимизацију) додатно би побољшао модуларност преводиоца. Процес генерисања излазног кода постао би независан од специфичности AST-а, што би олакшало додавање нових бекенд циљева, попут инфраструктуре за генерисање машинског кода (енг. *Low Level Virtual Machine*) или директног генерисања C кода.

Списак слика

Слика 1	Трансформација израза $5 + 2 * 3$ у AST на основу ВР-а оператора	4
Слика 2	Структура дизајнерског обрасца посетилац [11]	6
Слика 3	Поређење AST-а и HIR-а на примеру декларације доменског ентитета.	7
Слика 4	Процес серијализације модула и његовог увоза током превођења.	8
Слика 5	Теоријски приказ униформне (лево) и категоријалне расподеле (десно).	10
Слика 6	Комуникација између уредника текста и језичког сервера током уређивања документа [2]. ...	11
Слика 7	Структура дизајнерског обрасца диспечер команди.	13
Слика 8	GUI платформе <i>Mockaroo</i> за конфигурацију генератора података	18
Слика 9	Дијаграм архитектуре система	19
Слика 10	Ланац обраде у преводиоцу.	19
Слика 11	Пример рада семантичког анализатора.	22
Слика 12	Разрешавање симбола помоћу композитног кључа.	23
Слика 13	Архитектура LSP система и ток обраде LSP захтева.	25
Слика 14	Управљање интерним бафером током вишења унапред.	30
Слика 15	Декомпозиција текстуалног шаблона током синтаксне анализе.	31
Слика 16	Зависност времена извршавања фаза превођења од броја линија кода	40
Слика 17	Линеарна зависност времена превођења од броја модула за различите топологије.	41
Слика 18	Зависност меморијске потрошње од броја генерисаних записа.	42
Слика 19	Зависност времена генерисања од броја записа.	43
Слика 20	Зависност времена одзива LSP операција од величине датотеке.	44

Списак листинга

Листинг 1	Дефиниција граматике у алату <i>textX</i>	15
Листинг 2	Пример <i>JSON Schema</i>	16
Листинг 3	Шема поруке унутар <i>proto</i> датотеке	17
Листинг 4	Имплементација псеудослучајног генератора у језику <i>Python</i>	17
Листинг 5	Пример глобалних директива и увоза модула	20
Листинг 6	Дефинисање типова са ограничењима.	20
Листинг 7	Дефинисање колекцијских типова и вишедимензионалних структура	20
Листинг 8	Енумерација са факторима тежине расподеле	21
Листинг 9	Структурна композиција и механизам наслеђивања	21
Листинг 10	Референцирање вредности ради очувања интегритета	21
Листинг 11	Увоз модула и референцирање споља дефинисаног типа	23
Листинг 12	Пример генерисаног JSON-а.	24
Листинг 13	Спецификација шаблона и директива за генерисање података.	24
Листинг 14	Структура <i>TokenStream</i> компоненте.	30
Листинг 15	Структура и енумерација за репрезентацију језичких израза	32
Листинг 16	Енумерација <i>SymbolKind</i> за репрезентацију семантичких метаподатака	33
Листинг 17	Дефиниција семантичких грешака и мапирање у објекте дијагностике	34
Листинг 18	Инфраструктура за меморијску оптимизацију и мапирање изворног кода	34
Листинг 19	Структура <i>AstLowering</i> задужена за трансформацију AST-а у HIR	35
Листинг 20	Структуре података за глобално адресирање ставки	35
Листинг 21	Логика разрешавања, валидације кеша и релокације модула.	35
Листинг 22	Дефиниција особине <i>FileGenerator</i> за генерисање излазних формата.	36
Листинг 23	Ажурирање документа и синхронизација анализе	37
Листинг 24	Структура CLI команди	37

Списак табела

Табела 1	Пресликавање локалних идентификатора у глобални адресни простор током релокације.	8
Табела 2	Поређење предложеног решења са постојећим алатима	18
Табела 3	Преглед доступних команди корисничког интерфејса	26
Табела 4	Средње време извршавања фаза процеса превођења.	39
Табела 5	Време учитавања и разрешавања зависности у односу на топологију модула.	41
Табела 6	Поређење времена генерисања и меморијске потрошње различитих имплементација.	42

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (програмски интерфејс апликације)
AST	Abstract Syntax Tree (апстрактно синтаксно стабло)
BP	Binding Power (снага везивања)
CLI	Command Line Interface (интерфејс командне линије)
CSV	Comma-Separated Values (текстуални формат са вредностима одвојеним зарезом)
DSL	Domain Specific Language (језик специфичан за домен)
GPL	General Purpose Language (језик опште намене)
GUI	Graphical User Interface (графички кориснички интерфејс)
HIR	High-level Intermediate Representation (високонивојска међурепрезентација)
IR	Intermediate Representation (међурепрезентација)
JSON	JavaScript Object Notation (текстуални формат за размену података)
LSP	Language Server Protocol (протокол језичких сервера)
MIR	Medium-level Intermediate Representation (средњенивојска међурепрезентација)
PRNG	Pseudo-Random Number Generator (псеудослучајни генератор бројева)
RPC	Remote Procedure Call (позивање удаљених процедура)
SQL	Structured Query Language (структурирани језик за упите)
XML	Extensible Markup Language (прошириви језик за означавање података)

Списак коришћених појмова

Појам	Објашњење
Двоструко упућивање	Механизам избора методе у извршном времену на основу типова два објекта: објекта који прима позив и објекта који се прослеђује као аргумент.
Директива	Синтаксни конструкт који почиње симболом @. Служи за глобалну конфигурацију окружења, дефинисање метаподатака преводиоца и управљање модулима, обезбеђујући подешавање извршног окружења без измене логике самих шаблона.
Засебно превођење	Метод превођења програмског кода где се изворне датотеке преводе независно, чиме се убрзава развој јер нема потребе за поновним превођењем целог система.
Инкрементално превођење	Техника поновног превођења само измењених делова кода ради скраћивања времена изградње система.
Интернирање стрингова	Техника оптимизације меморије где се јединствени текстуални низови складиште у базен и мењају нумеричким кључевима ради бржег поређења.
Језички сервер	Системска компонента која анализира код и омогућава навигацију. Састоји се од комуникационог, аналитичког и функционалног слоја, а служи за пружање повратних информација, укључујући дијагностику грешака и контекстуално допуњавање кода.
JSON-RPC	Протокол за позивање удаљених процедура који користи JSON формат за кодирање порука, коришћен у LSP комуникацији.
Међурепрезентација	Интерни модел или структура података коју преводилац користи за представљање изворног кода између фазе синтаксне анализе и генерисања кода.
Повезивање	Процес спајања независних модула у извршну целину.
Релокација	Корак током повезивања који ажурира нумеричке референце и меморијске адресе унутар учитаног кода.
Самоописујући формат	Формат записа који не захтева унапред дефинисану шему, већ унутар записа чува метаподатке о типовима и структурама поља.
Серијализација	Процес превођења меморијских структура података у линеарни низ бајтова погодан за трајно чување на диску или пренос мрежом.
Спуштање	Процес сукцесивне трансформације међурепрезентације којим се сложени језички конструкти свode на једноставније, каноничке форме ближе извршном коду.
Табела симбола	Структура података коју преводилац користи за мапирање идентификатора са својствима, опсезима видљивости и локацијама.
Шаблон	Активна семантичка јединица која се користи за моделирање објеката над којима се извршава генерација података.

Биографија

Лазар Нагулов је рођен 30. априла 2003. године у Сомбору. Основну школу „Доситеј Обрадовић” завршио је у родном граду. Након тога уписао је гимназију „Вељко Петровић” у Сомбору, смер за ученике са посебним способностима за рачунарство и информатику, коју је завршио 2022. године. Исте године уписао је Факултет техничких наука Универзитета у Новом Саду, смер софтверско инжењерство и информационе технологије.

Литература

- [1] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2011.
- [2] Microsoft, “Language Server Protocol Specification (Version 3.17).” Accessed: April 29, 2026. [Online]. Доступно на <https://microsoft.github.io/language-server-protocol>
- [3] M. Voelter, *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. dslbook.org, 2013.
- [4] R. Rodriguez-Echeverria, J. L. C. Izquierdo, M. Wimmer, and J. Cabot, “Towards a Language Server Protocol Infrastructure for Graphical Modeling,” in *Proceedings of the 21st ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2018, pp. 370–380.
- [5] D. Bork and P. Langer, “Language Server Protocol: An Introduction to the Protocol, Its Use, and Adoption for Web Modeling Tools,” *Enterprise Modelling and Information Systems Architectures (EMISAJ)*, vol. 18, p. 9:1–9:31, 2023.
- [6] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007.
- [7] T. Parr, *Language Implementation Patterns: Create Your Own Domain-specific and General Programming Languages*. in Pragmatic Bookshelf Series. Pragmatic Bookshelf, 2010. [Online]. Доступно на <https://books.google.rs/books?id=C7xuPgAACAAJ>
- [8] T. Ball, *Writing An Interpreter In Go*. Wainshain Press, 2016.
- [9] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann, 1997.
- [10] The Rust Project Developers, “Guide to rustc Development.” Accessed: July 7, 2026. [Online]. Доступно на <https://rustc-dev-guide.rust-lang.org/>
- [11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. in Addison-Wesley Professional Computing Series. Addison-Wesley, 1994.
- [12] R. Nystrom, *Crafting Interpreters*. Genever Benning, 2021. [Online]. Доступно на <https://craftinginterpreters.com/>
- [13] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [14] S. Furuhashi, “MessagePack Specification.” Accessed: July 7, 2026. [Online]. Доступно на <https://msgpack.org/>
- [15] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY: McGraw-Hill Education, 2014.
- [16] H. Abelson and G. J. Sussman, *Structure and Interpretation of Computer Programs*, 2nd ed. Cambridge, Massachusetts: MIT press, 1996.
- [17] J. Hughes, “Why functional programming matters,” *The computer journal*, vol. 32, no. 2, pp. 98–107, 1989.
- [18] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.
- [19] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. New York, NY: Cambridge University Press, 2007.

-
- [20] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [21] H. Bänder, “Decoupling Language and Editor – The Impact of the Language Server Protocol on Textual Domain-Specific Languages,” in *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, 2019, pp. 129–140.
- [22] E. S. Raymond, *The Art of UNIX Programming*. Addison-Wesley, 2003.
- [23] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [24] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall, 2002.
- [25] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. John Wiley & Sons, 1996.
- [26] I. Dejanović, R. Vadera, G. Milosavljević, and Ž. Vuković, “textX: A Python tool for Domain-Specific Languages implementation,” *Knowledge-Based Systems*, vol. 115, pp. 1–4, 2017, doi: [10.1016/j.knosys.2016.10.023](https://doi.org/10.1016/j.knosys.2016.10.023).
- [27] B. Ford, “Parsing expression grammars: a recognition-based syntactic foundation,” in *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2004, pp. 111–122. doi: [10.1145/964001.964011](https://doi.org/10.1145/964001.964011).
- [28] F. Campagne, *The MPS Language Workbench: Volume I*, 3rd ed. CreateSpace Independent Publishing Platform, 2016.
- [29] M. Voelter, J. Siegmund, T. Berger, and B. Kolb, “Towards user-friendly projectional editors,” *Software & Systems Modeling*, vol. 13, no. 4, pp. 1617–1645, 2014, doi: [10.1007/s10270-013-0329-0](https://doi.org/10.1007/s10270-013-0329-0).
- [30] A. Wright, H. Andrews, and B. Hutton, “JSON Schema: A Media Type for Describing JSON Data Formats,” technical report, 2022.
- [31] G. LLC, “Protocol Buffers Developer Guide.” 2026.
- [32] F. Contributors, “Faker JavaScript/Python Library Documentation.” 2026.
- [33] M. LLC, “Mockaroo: Realistic Data Generator and API Mocking Tool.” 2026.
- [34] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [35] Rust Project Developers, “The Rust Standard Library Documentation.” 2026.
- [36] dani0595, “rmp-serde: A Rust MessagePack serializer/deserializer.” [Online]. Доступно на <https://github.com/3Hren/msgpack-rust>
- [37] E. Kalderon and contributors, “tower-lsp: Language Server Protocol implementation based on Tower.” 2025.
- [38] K. B. Knapp and T. C. Community, “clap.” [Online]. Доступно на <https://docs.rs/clap>
- [39] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1993.